

FastSAR: High-Throughput, High-Fidelity Open-Source SAR Image Formation on CPUs, GPUs and TPUs

Paul Singerman
Penn State University
pgs16@psu.edu

Tom Braum
Penn State University
trb5864@psu.edu

October 2026

Abstract

Exact backprojection of a spaceborne SAR collection needs one interpolated read per pixel and pulse, 1.6 trillion for a 12,207 by 8,808 pixel Umbra image of the Panama Canal. ISCE3, the fastest of the open-source backprojections evaluated here that produce an image of this collection, would take an estimated 4.8 h on a 16-vCPU instance. FastSAR is an open-source library with backprojection kernels for CPUs, NVIDIA GPUs and Google TPUs. Its factorized backprojection forms the image in 11.3 s on that instance, 3.8 s on an NVIDIA L4, and 5.2 and 3.1 s on Google TPU v5e and v6e. The errors of its float32 and float16 images in three regions are -56 to -63 dB relative to a float64 exact backprojection. Its own exact backprojection, 24 s on the L4, is 19 to 21 dB more accurate than its float32 image. At float32 accuracy (three-pass bfloat16 products on the TPUs) a thousand images cost \$0.75 on the L4, \$1.32 to \$1.58 on the TPUs and \$3.04 on the CPU. On five Capella collections, including a sliding spotlight and two stripmaps, FastSAR’s worst-region error is -59.1 to -61.5 dB, 10 to 34 dB below ISCE3’s when both are scored with the per-pulse antenna positions of the phase-history files.

1 Introduction

Image formation turns a SAR phase history of 10^8 to 10^9 complex samples into an image with a comparable number of pixels (Table 1). Exact backprojection makes no approximation to the imaging geometry [1, 2], and its cost scales with the product of the pixel and pulse counts; for the Panama collection of Table 1 that product is 1.6 trillion interpolated reads. The three open-source backprojections that produce an image of this collection, RITSAR, the Air Force Research Laboratory (AFRL) toolbox and ISCE3, would take an estimated 4.8 to 69 h on a 16-vCPU instance (Table 3).

FastSAR is an open-source SAR image formation library with one Python interface for CPUs, NVIDIA GPUs and Google TPUs (Section 3). On the Panama collection it forms a float32-class image (float32 arithmetic or its TPU equivalent, Section 4) 1,500 times faster than ISCE3’s estimate on the same CPU instance and 430 times faster on an NVIDIA L4 (Table 3).

Its error in the lock, port and ship regions is 3.9 to 6.7 dB below the lowest open-source error, that of the AFRL toolbox’s `bpBasic`, and about 25 to 30 dB below ISCE3’s off-center patches (Section 5.1). The other open-source backprojections are slower still, by estimated factors of up to 22,000 (Table 3). A thousand images cost \$0.75 on the L4 (Table 4). FastSAR’s exact backprojection, which has separate CPU and CUDA kernels, lowers the error by a further 19 to 21 dB (Section 5.2) at 38 times the factorized time on the CPU and 6.4 times on the L4 (Table 4).

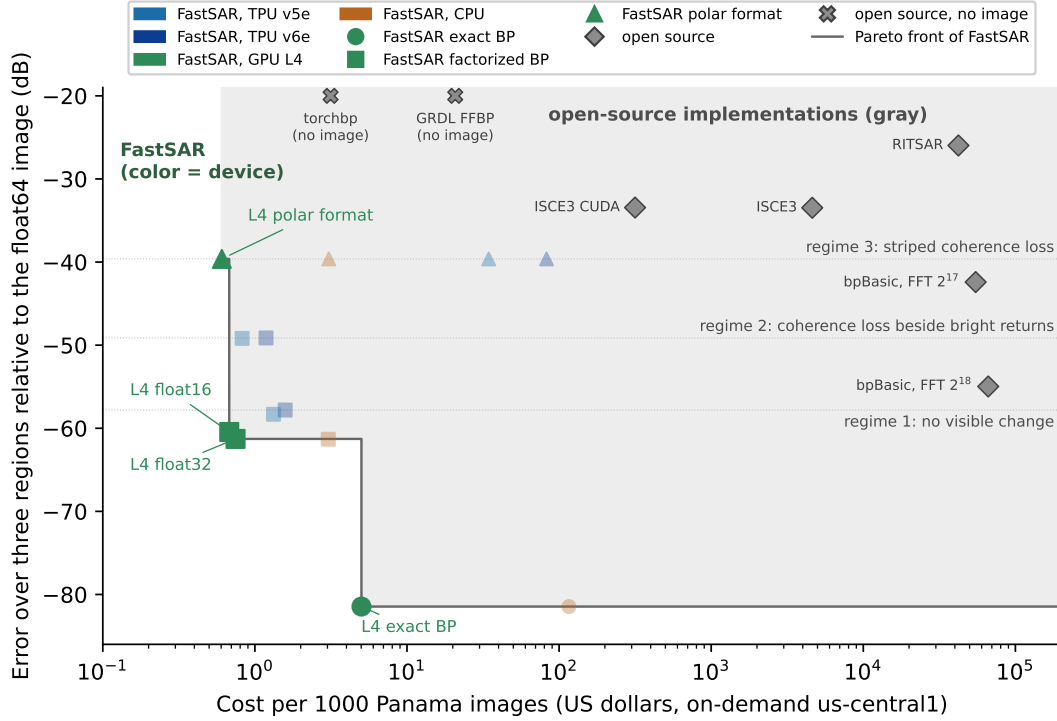


Figure 1: Cost per thousand Panama Canal images against the error over the lock, port and ship regions of Figure 3, for FastSAR on each device and for the open-source implementations on the same CPU instance type and L4 GPU (gray). The three regions are pooled with weights equal to their reference energy. FastSAR’s errors are relative to the 64-times computation of Section 4, the open-source implementations’ relative to the reference; ISCE3 is scored as the mean over its own three patches. Times marked est. are estimates (Section 5.1). The torchbp and GRDL backprojections, which produced no usable image (amplitude correlation with the reference 0.22 or less, Appendix G), are drawn off scale. The line is the Pareto front of the FastSAR configurations on these pooled region errors (the bold rows of Table 4 use whole-image errors); those off the front are drawn faded and unlabeled (Table 4 identifies each configuration). Each dotted line marks the worst FastSAR configuration of one of the image-quality regimes 1 to 3 of Section 5.2.

The factorized algorithm is written as tile-wise phase rotations and decimation filters followed by dense matrix products, with a kernel for each device (Section 3). In the profiled builds each stage runs within 1.0 to 2.5 times a lower bound set by the hardware unit that limits it (Table 9, Appendix D).

Of the four devices evaluated (two TPU generations, one GPU and one CPU instance, Table 2), the L4 forms the float32-class image at the lowest cost (Table 4). Reduced precision, float16 throughout on the L4 and single-pass products on the TPUs, lowers the cost by 9% and by 25 to 38% and raises the error by 1.0 and 12 dB (Sections 5.2 and 5.3). On five Capella collections in three modes the float32-class errors stay below -59 dB (Table 5).

2 Related work

Backprojection on parallel hardware has been reported since 2009 on FPGAs [3], many-core CPUs [4] and GPUs. On GPUs, single-GPU CUDA implementations [5], gigapixel formation on GPU clusters [6], nonuniform FFT interpolation [7] and float16 arithmetic [8] have been described. Fast factorized backprojection follows the quadtree of McCorkle and Rofheart [9] and the formulation of Ulander, Hellsten and Stenström [10]; it has been applied to nonlinear tracks [11] and to circular SAR on GPUs [12]. Polar format is described in [13, 1]; Jakowatz and Doren compared it with backprojection [14], and Doerry [15] and Gorham and Rigling [16] derived its scene-size limit.

Among open-source implementations, RITSAR [17], GRDL [18] and ISCE3 [19] run on the CPU, as does the AFRL toolbox [2], distributed by the National Geospatial-Intelligence Agency, with the backprojection `bpBasic` and the polar format `pfa_mem`. On the GPU there are `torchbp` [20] and ISCE3’s CUDA version. Outside machine learning the TPU [21, 22] has been used for Fourier transforms [23, 24] and linear algebra [25]. No published TPU implementation of SAR image formation, and no comparison of the open-source implementations on matched hardware against a common float64 reference, was found.

3 FastSAR

FastSAR takes a phase history, the antenna positions and an output grid and returns a complex image. The CPU, CUDA and TPU backends share the host-side float64 geometry and the factorization plan, including its decimation filters, and stripmap and sliding-spotlight collections are formed as mosaics of factorized patches. Its three algorithms are described in Appendix A: factorized backprojection (factorized BP in the tables) follows [9, 10]. Exact backprojection (exact BP) [2] reads range profiles, oversampled 8 times by default from release 0.1.1 (4 times in 0.1.0), with cubic interpolation. Polar format [13, 1] ends with a resampling that removes its planar-wavefront displacement (Appendix E).

The factorized kernels are written in C++ with OpenMP for the CPU, in CUDA for NVIDIA GPUs and in Pallas, JAX’s kernel language, for TPUs (the fused kernels of the tables; Appendix D). They replaced a pure JAX implementation (the JAX program), which remains in the library and is timed on the CPU in the release records. In device time the kernels formed the float32-class Panama image 1.9, 2.0 and 3.9 times faster than the JAX program on the v6e, v5e and L4. On the CPU the released C++ kernels form it 20 times faster than the JAX program (Table 8).

The precision options are float32 or float16 throughout (float16 data and tensor-core operands, float32 accumulation) on the GPU and three-pass or single-pass bfloat16 products on the TPU (Appendix B). Exact backprojection has no TPU kernel; on a TPU it runs in pure JAX, at about 15 and 49 h per Panama image on the v5e and v6e (Appendix C).

4 Data and protocol

Three spotlight collections from Umbra’s open data catalog [26] are used throughout (Table 1, Figure 2): the Pacific entrance of the Panama Canal (2023-07-18), central Melbourne (2023-02-08) and farmland near Johnston, Iowa (2023-10-20). Each was processed from the Compensated Phase History Data (CPHD) file as delivered onto the vendor’s image grid, on the four devices of Table 2: c4d-highmem-16 instances with 8 cores and 16 vCPUs (the CPU instance), an NVIDIA L4, and TPU v5e and v6e.

The four Panama regions of Figure 3 were identified against Sentinel-2 imagery (Appendix I); the lock, port and ship regions, the three scored regions, are used for the comparison with open-source

Table 1: The three collections and their native image grids. Spacings are azimuth by range in the slant plane; squint is the Doppler cone angle less 90° at the aperture center; range is the slant range to the scene center at the start of the aperture.

Scene	Image (az $\times$ rg)	Spacing (m)	Pulses $\times$ samples	Bandwidth (MHz)	Grazing angle ($^\circ$)	Squint ($^\circ$)	Range (km)
Panama Canal	$12,207 \times 8,808$	0.413×0.356	$15,186 \times 14,399$	337	42.6	-28.1	738
Melbourne	$10,077 \times 10,273$	0.422×0.298	$15,948 \times 14,399$	405	43.4	0.6	760
Iowa farmland	$8,521 \times 8,465$	0.517×0.328	$13,003 \times 11,663$	356	50.3	2.8	653

Table 2: Instances, the arithmetic timed on each (Appendix B) and on-demand prices (US dollars per hour, us-central1).

Device	Instance	Arithmetic timed	Price (\$/h)
TPU v5e	1 chip (v5litepod-1), 24 vCPU host	bfloat16 single- and three-pass products; float32 data	1.200
TPU v6e	1 chip (v6e-1), 44 vCPU host	bfloat16 single- and three-pass products; float32 data	2.700
L4	g2-standard-4, 1 NVIDIA L4 (24 GB)	float32; float16 throughout	0.707
CPU	c4d-highmem-16, AMD EPYC 9B45, 8 cores, 126 GB	float32	0.964

implementations. Five Capella collections [27] extend the evaluation to a second vendor and to other modes (Section 5.4): two spotlights (2024-10-04 and 2025-03-03), a sliding spotlight (2022-09-30; Capella’s mode SS) and two stripmaps (2021-11-12 and 2025-12-08).

Images are compared, after one fitted gain, with the reference, a float64 exact backprojection of the same collection with range profiles oversampled 16 times. Once the vendor’s image is resampled through the planar-wavefront displacement model, the reference agrees with it in position to within 0.25 pixel root-mean-square (Appendix F). The error is the energy of the difference relative to that of the reference, in dB, over the whole image unless stated otherwise (Appendix H defines the other statistics). The reference is converged to about -67.3 dB, so the region errors of FastSAR in Table 3 and Figure 1 are measured against a second float64 computation with profiles oversampled 64 times, the 64-times computation (Appendix H).

Time is the wall-clock time to form one image on a warm device, from the phase history in host memory to the image in host memory, transfers and per-collection host work included, compilation and disk reads excluded. The shorter of two warm calls is reported; it agrees with the time per image of a back-to-back loop within 6% on the CPU and the L4, so differences below 6% between rows are not resolved. Cost per image is the hourly price divided by the loop’s throughput or, where no loop was run, the price multiplied by the time.

Float32-class accuracy means float32 arithmetic on the CPU and L4 and three-pass products on the TPUs. Each configuration is the library’s default for its backend and precision option, through the public interface of release 0.1.0 or of a release candidate; Appendix H states the differences, the loop mechanics and the tuning status of each algorithm.

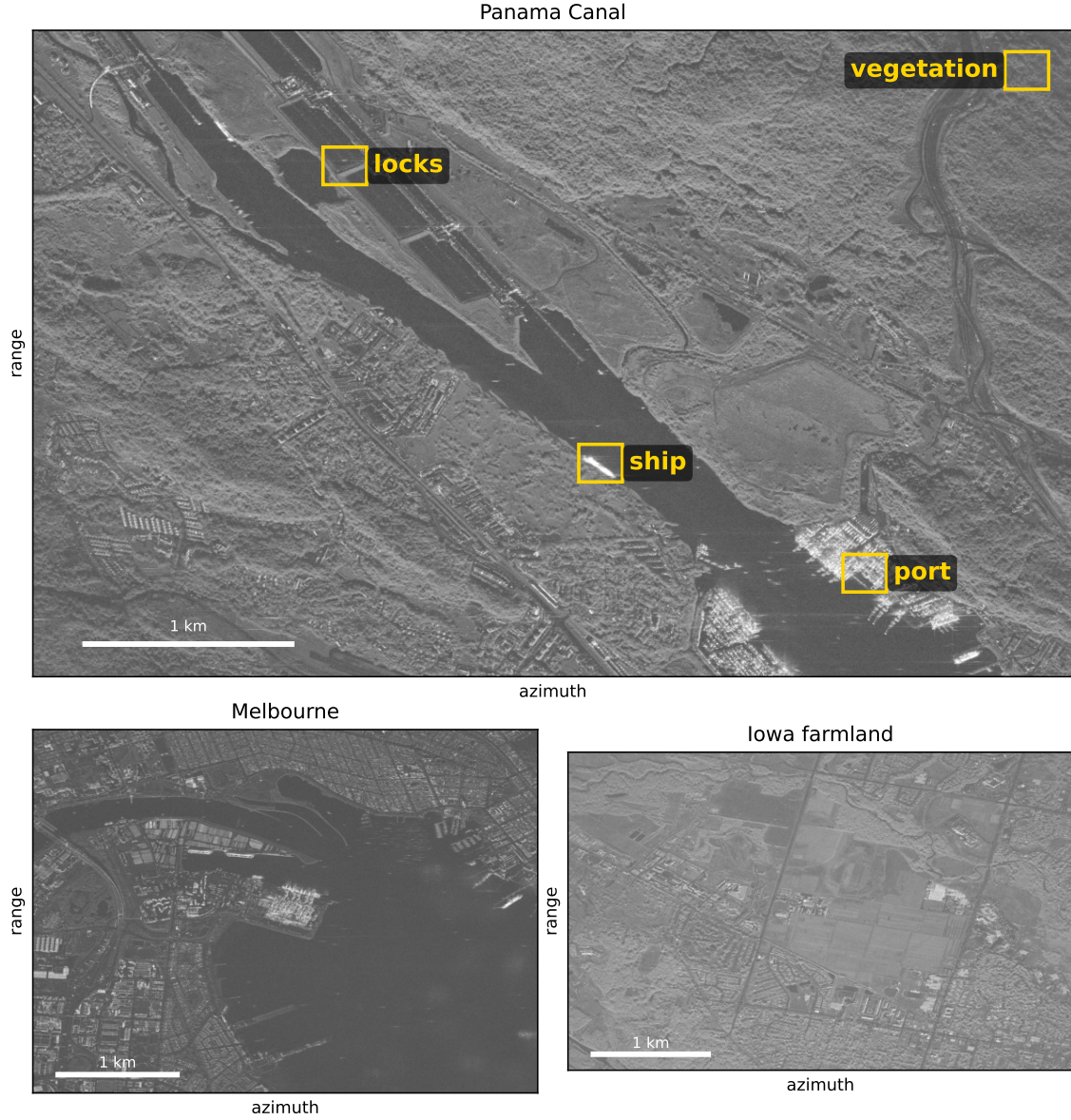


Figure 2: The three collections as formed by the reference (Section 4), downsampled for display (amplitude in decibels over a 45 dB range, azimuth horizontal, range vertical, 1 km bar). The yellow boxes on the Panama Canal image are the four 512 by 512 pixel regions of Figure 3.

5 Results

5.1 Comparison with open-source implementations

Of the open-source backprojections evaluated here, ISCE3 is the fastest that produced an image. It would need an estimated 4.8 h for the Panama image on the CPU and 27 min on the L4, against 11.3 s and 3.8 s for FastSAR’s factorized backprojection (Table 3). After removal of a fitted linear phase ramp, ISCE3 reaches -54.9 dB on the CPU at the scene center and -31.5 to -31.9 dB off center (Appendix G).

The lowest open-source error on the three scored regions is that of `bpBasic` with a 2^{18} -point FFT. Scored against the reference, it is 3.9, 4.7 and 6.7 dB above FastSAR’s float32 factorized image on the CPU in the lock, port and ship regions (Table 3 lists FastSAR against the 64-times computation, so its FastSAR rows are lower than this comparison by the reference’s own error).

The two open-source polar-format implementations, `pfa_mem` (44.5 s) and GRDL’s polar format (24.5 s), leave the planar-wavefront displacement uncorrected. Their log-amplitude correlations with the reference, 0.80 to 0.89 and 0.78 to 0.93, are below the 0.996 to 0.999 of FastSAR’s polar format (Table 12).

FastSAR’s exact backprojection forms the full image in 433 s on the CPU and 24 s on the L4, 40 and 67 times faster than the ISCE3 estimates, with region errors of -77.5 to -81.7 dB. Scaling by pixel count overpredicted the full-image time of FastSAR’s exact backprojection on the CPU by a factor of 3.9 (28 min predicted, 433 s measured), so the open-source estimates and the speed ratios derived from them should be read as upper bounds.

5.2 Image quality

In addition to the energy ratio, each image is assigned to one of three regimes by its amplitude deviation and its coherence with the reference over 5 by 5 pixel windows (the 5 by 5 coherence below). These local statistics are sensitive to isolated errors that do not change the energy ratio. Regime 1 requires an amplitude deviation below 0.05 dB at the 99th percentile and a coherence above 0.999 at the 0.1 percentile. In regime 2 the coherence falls below 0.999 only beside bright returns, and in regime 3 it falls below 0.99 in stripes (the underlying statistics are in Table 15).

On Panama only single-pass TPU products fall in regime 2 and only polar format in regime 3; the other configurations meet the regime 1 criteria (Figures 3 and 4, Table 15). The errors of the float32-class images are -59.9 dB on the L4, -60.0 dB on the CPU, and -57.8 and -57.9 dB with three-pass products on the v6e and v5e. The JAX program on the CPU, which like the TPU kernels evaluates the last level from float32 geometry, gives -58.3 dB, which suggests that the float32 last level accounts for most of the TPU images’ deficit of up to 2.2 dB.

Relative to the float32-class images, float16 throughout on the L4 adds 1.0 dB of error and single-pass products on the TPUs add 12 dB, increases that match the 11 and 8 significant bits of the two formats (Appendix B). The phase deviation of the single-pass v6e image is 1.1° at the 99th percentile but reaches 106° beside bright returns (173° on Melbourne); its effect on interferometric products was not evaluated. The polar-format error is -32.1 dB, growing from -40 dB in the inner ring to -30 dB in the outer ring; the cause of this residual was not identified (Appendix E).

The error of FastSAR’s exact backprojection, with cubic interpolation of profiles oversampled 8 times, is -77.5 to -81.7 dB in the three scored regions against the 64-times computation, 19 to 21 dB below the float32 factorized image. Over the whole image it is -67.4 dB against the reference, the reference’s limit; Appendix H gives the 4-times and linear variants.

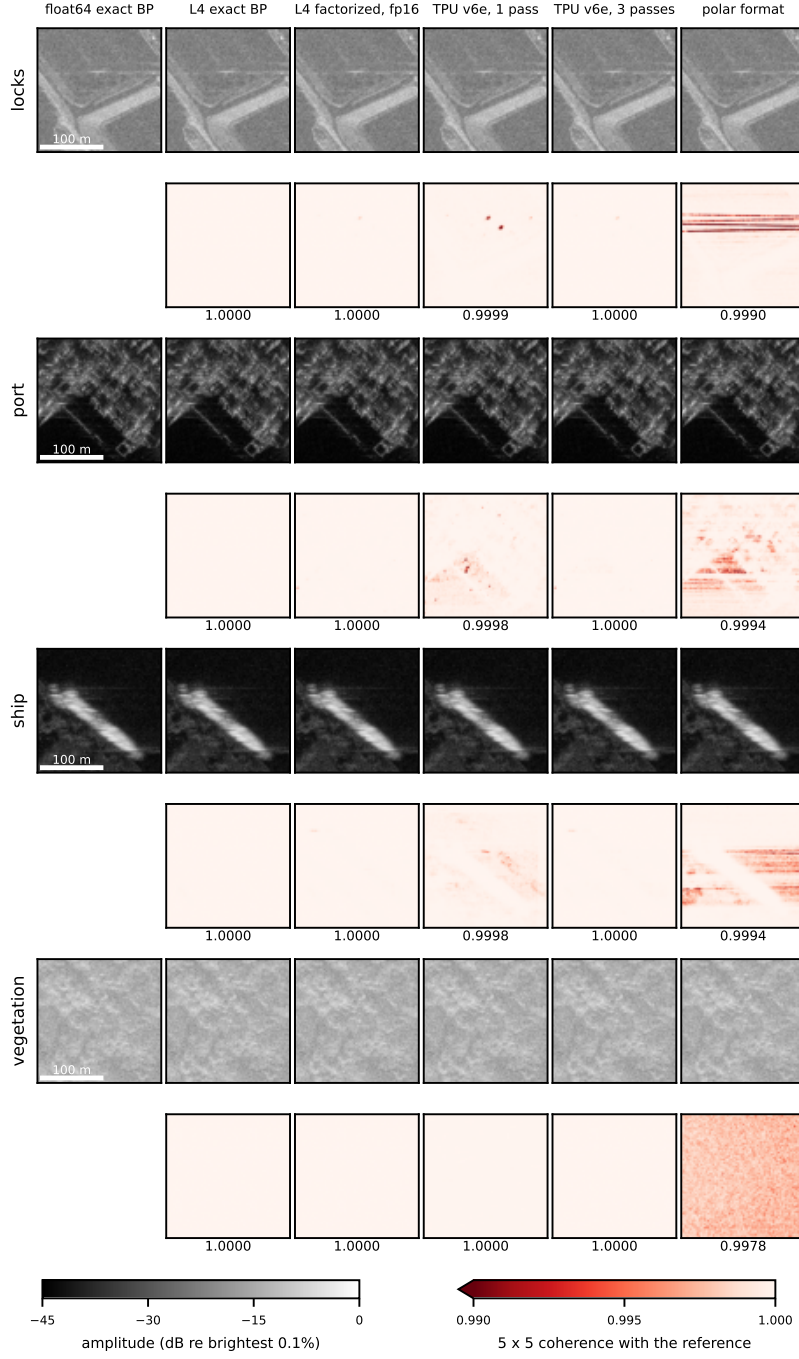


Figure 3: Panama Canal, four regions of 512 by 512 pixels at full resolution (100 m bar). Top of each pair: amplitude over a 45 dB range for the reference, exact backprojection on the L4 (cubic, $8\times$), factorized backprojection on the L4 in float32 and in float16 throughout, the TPU v6e factorized image with single-pass and three-pass products, and polar format on the L4. Bottom: the 5 by 5 coherence of each image with the reference from 0.99 (red) to 1 (white), with its mean below each panel. The regions are the Cocolí Locks, the Port of Balboa, a ship in the channel and vegetated terrain 2.7 km from the scene center; Appendix I locates them.

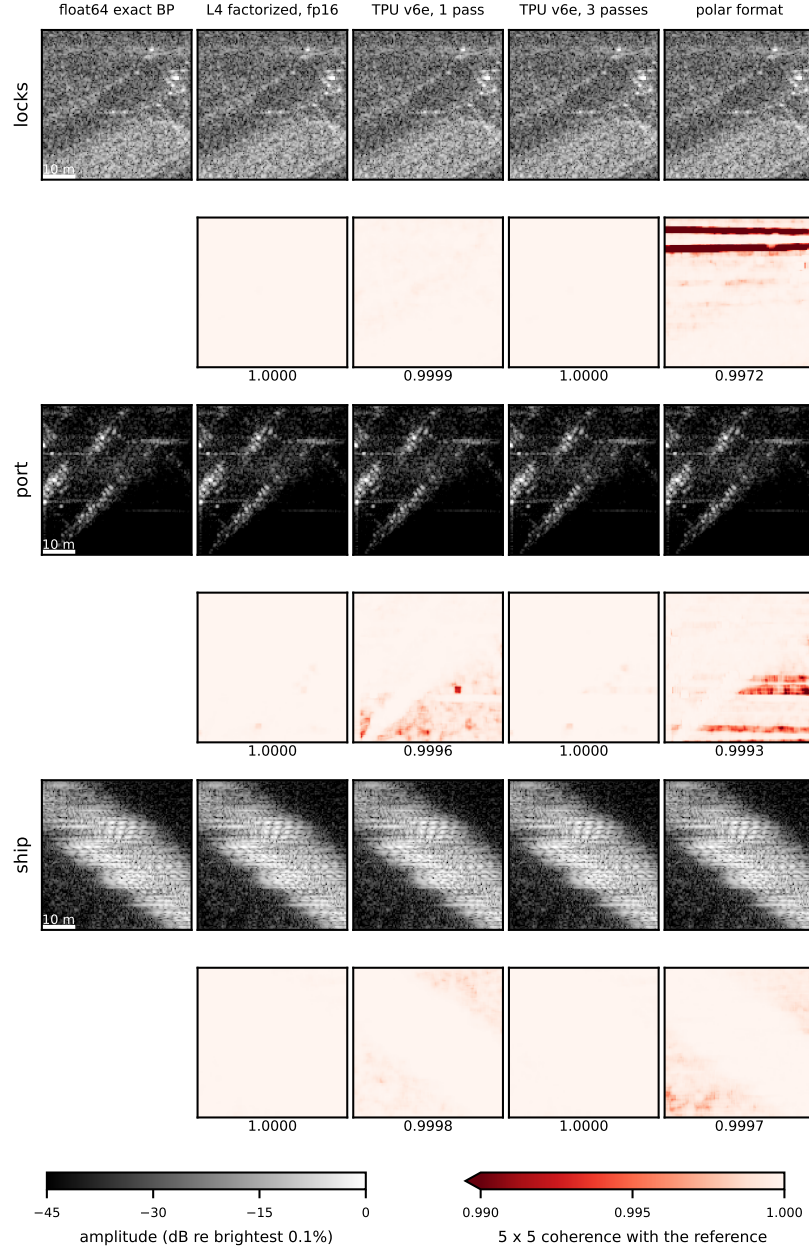


Figure 4: Panama Canal at pixel scale: 128 by 128 pixel windows (53 m in azimuth by 46 m in slant range, 10 m bar) around the brightest feature of the lock, port and ship regions of Figure 3, drawn without interpolation (pixel spacing 0.41 by 0.36 m). Columns, scales and coherence maps as in Figure 3, without the L4 exact backprojection.

Table 3: Error against the reference and time per full image on the Umbra Panama collection. Errors (dB, Section 4) are the range over the lock, port and ship regions of Figure 3, worst to best. ISCE3 was scored instead on three 256 by 256 pixel patches of its own grid (the scene center and two off-center positions), after removal of a linear phase ramp fitted to each. An error of 0.0 dB means that the difference has the same energy as the reference. FastSAR’s errors are measured against the 64-times computation, those of the open-source implementations against the reference (Section 4). $16\times$, $8\times$ and $2\times$ give the oversampling of the range profiles; FastSAR’s exact backprojection interpolates them cubically, the others linearly. Times are measured, except those marked est., which scale the time on the scored regions by the number of pixels (Section 5.1).

Implementation	Instance	Error (dB)	Time per full image
RITSAR, as published, $16\times$	CPU	−25.3 to −27.3	44 h (est.)
bpBasic, FFT 2^{17} (default)	CPU	−38.1 to −43.2	57 h (est.)
bpBasic, FFT 2^{18}	CPU	−52.9 to −55.4	69 h (est.)
ISCE3 backproject	CPU	−31.5 to −54.9	4.8 h (est.)
GRDL, factorized	CPU	no image [†]	77.5 s
FastSAR, exact, $8\times$	CPU	−77.5 to −81.7	433 s
FastSAR, factorized, float32	CPU	−57.2 to −62.6	11.3 s
ISCE3 backproject, CUDA	L4	−31.5 to −52.2	27 min (est.)
torchbp, exact, $2\times$	L4	no image* (0.0)	16.0 s
torchbp, factorized	L4	no image*	out of memory
FastSAR, exact, $8\times$	L4	−77.5 to −81.7	24 s
FastSAR, factorized, float32	L4	−57.1 to −62.6	3.8 s
FastSAR, factorized, float16 throughout	L4	−55.6 to −61.6	3.5 s

*torchbp evaluates the pixel-to-antenna distance in float32. At the 744 km mid-aperture range of this collection (Table 1 gives the range at the start of the aperture) the float32 spacing is 6 cm, about two wavelengths, so the phase of each pulse is lost. A float64 evaluation of the same formula forms the image. On the three regions alone, where memory suffices, its factorized backprojection returns an empty image for the same reason.

[†]GRDL’s factorized backprojection is a stripmap algorithm; its image of this spotlight collection is smeared in cross-range (Appendix G).

5.3 Cost per image on CPU, GPU and TPU

At float32-class accuracy the cost of a thousand Panama images (Table 4) rises from the L4 (\$0.75) through the v5e (\$1.32) and the v6e (\$1.58) to the CPU (\$3.04). The order is the same on Melbourne and Iowa, where the L4 costs \$0.78 and \$0.50 and the CPU \$3.38 and \$2.02 (Table 14). The v6e is the fastest device at this accuracy, 3.1 s per image where the L4 takes 3.8 s, at 3.8 times the L4’s hourly price.

Relative to the float32-class configuration on the same device, float16 throughout costs 9% less on the L4, a margin only slightly above the 6% timing resolution of Section 4, and single-pass products cost 25 and 38% less on the v6e and v5e. Polar format on the L4 costs \$0.61 per thousand images. On the TPUs it takes 20 to 35 times as long as the three-pass factorized configuration, because its resampling steps are scattered gathers, which run on the TPU’s vector unit at about 4×10^7 reads per second (Appendix C). Exact backprojection on the L4 costs \$5.02 per thousand images, 6.7 times the float32 factorized cost.

Table 4: Panama Canal: the configurations of release 0.1.0, cheapest first. Bold rows are the Pareto front of the whole-image errors, the configurations that no other configuration beats on both cost and error (errors within 0.05 dB before rounding counted as equal). Time is the single-image time; cost is computed from the throughput of the timed loop, which on the TPUs overlaps staging with formation; error is the whole-image error (Section 4); an asterisk marks rows timed without a loop, with cost computed from the single-image time. Energy is that of the processor alone over the per-image time of the cost column; its basis on each device is given with Table 13. Table 14 gives the other two collections.

Device	Algorithm	Arithmetic	Time (s)	\$ per 1000	kWh per 1000	Error (dB)
L4	Polar format	float32, gather	3.0	0.61	0.04	−32.1
L4	Factorized BP	float16 throughout, CUDA kernels	3.5	0.68	0.06	−58.9
L4	Factorized BP	float32, CUDA kernels	3.8	0.75	0.07	−59.9
TPU v5e	Factorized BP	single-pass products, fused kernels	3.6	0.82	0.08 to 0.14	−46.2
TPU v6e	Factorized BP	single-pass products, fused kernels	2.6	1.18	0.09 to 0.15	−46.2
TPU v5e	Factorized BP	three-pass products, fused kernels	5.2	1.32	0.13 to 0.22	−57.9
TPU v6e	Factorized BP	three-pass products, fused kernels	3.1	1.58	0.12 to 0.21	−57.8
CPU	Factorized BP	float32, C++ kernels	11.3	3.04	0.08	−60.0
CPU	Polar format	float32, gather	11.4	3.06	0.08	−32.1
L4	Exact BP	linear interpolation, 8× oversampled	20.3	3.93	0.40	−56.9
L4	Exact BP	cubic interpolation, 8× oversampled	24.1	5.02	0.51	−67.4
TPU v5e	Polar format	float32, gather	103.4	34.45	3.44 to 5.74	−32.1
TPU v6e	Polar format	float32, gather	110.2	82.69	6.12 to 10.72	−32.1
CPU	Exact BP	cubic interpolation, 8× oversampled	433.5	116.08*	3.01	−67.4

5.4 Other collections and modes

On the Umbra collections of Melbourne and Iowa, the errors of the factorized configurations are within 1.1 dB of the Panama values (supplementary tables, Appendix J). On the five Capella collections (Table 5), FastSAR’s worst-region error over the three devices lies between −59.1 and −61.5 dB.

On the CPU instance, ISCE3’s backprojection would take an estimated 36 to 3,600 times as long as FastSAR. The two differ in the receive model (Appendix A). Against the reference formed with the CPHD positions, ISCE3’s worst-region error is −25.8 to −50.6 dB, 10 to 34 dB above FastSAR’s. Scored against a reference formed with its own receive model, ISCE3’s worst-region error is 4.0 to 16.6 dB below FastSAR’s worst error over the three devices on the 2024 spotlight, the 2025 spotlight and the 2021 stripmap.

GRDL’s frequency-domain processors, whose slant-plane images were not scored against the reference, were faster than FastSAR on the CPU for both stripmaps and slower for both spotlights. Its range-Doppler processor formed the 2021 stripmap in 27.7 s, compared with 47.5 s for FastSAR on the CPU. Among the FastSAR devices, the v6e has the shortest time on all five collections. The L4 costs less per image than the v6e on all collections except the 2025 spotlight; a thousand images of the 2025 stripmap cost \$95 on the L4, \$130 on the v6e and \$234 on the CPU.

Table 5: The five Capella collections. FastSAR time per image (host memory to host memory, Section 4) at float32-class accuracy and cost per thousand images (hourly price times the single-image time) on the CPU instance, the L4 and the v6e; the L4 runs used a g2-standard-32 instance at \$1.734 per hour. * Run with smaller first-level groups (with a `MemoryWarning`), because full speed needs more than the roughly 23 GB free on the L4: 33.8 GB for the 2024 spotlight, 23.3 GB for the 2025 one. FastSAR error: the worst of three 512 by 512 pixel regions against the float64 reference over the three devices. ISCE3: whole-image time estimated from the three regions on the CPU instance, and worst-region error against a reference formed with ISCE3’s receive model (Section 5.4) and against the reference with the per-pulse antenna positions of the CPHD file (CPHD), as for FastSAR. GRDL: algorithm (PFA, polar format; RDA, range-Doppler; SMPFA, GRDL’s stripmap polar format) and time on the CPU instance; its slant-plane images are not scored against the reference.

Collection	Pulses	Mpx	FastSAR time (s)			FastSAR cost (\$/1000)			FastSAR error (dB)	FastSAR time (h)	ISCE3			GRDL	
			CPU	L4	v6e	CPU	L4	v6e			own model (dB)	CPHD (dB)		alg.	time (s)
Spotlight, 2024	74,203	258	54.0	18.0*	12.3	14	8.7*	9.2	-59.1	54.1	-63.1	-25.8	PFA		99
Spotlight, 2025	54,267	255	46.1	17.9*	9.8	12	8.6*	7.4	-59.2	41.6	-75.7	-36.3	PFA		70
Sliding spotlight, 2022	123,799	72	286	57.7	55.3	77	28	41	-61.5	2.9	-61.3	-38.6	-		-
Stripmap, 2021	39,898	83	47.5	11.5	9.6	13	5.5	7.2	-60.2	2.3	-64.8	-50.6	RDA		28
Stripmap, 2025	237,720	1,132	875	197	173	234	95	130	-60.6	48.2	-59.2	-26.9	SMPFA		550

6 Conclusions

On the Panama collection a commodity GPU, the L4, forms a float32-class spotlight image for \$0.75 per thousand images, and the two TPU generations and the CPU instance cost 1.8 to 4.1 times as much (Table 4). The ordering holds on the Melbourne and Iowa collections and, except for the 2025 spotlight, on the Capella collections. The time margins over the open-source implementations, 40 to 22,000 times, are large compared with the factor of about four by which the pixel-count scaling was found to overpredict a full-image time (Section 5.1).

The v6e is fastest on every collection but, at 3.8 times the L4’s hourly price, costs more per image on all but one. On each device the factorized algorithm’s stages run within 1.0 to 2.5 times the bounds of their limiting units (Table 9). The kernel improvements still identified in the profiles are about 12% on the v6e and 10% on the L4 (Appendix D). Gains beyond this would have to come from a change to the factorization plan itself, whose effect on the bounds was not measured.

Exact backprojection is 19 to 21 dB more accurate than the factorized algorithm in the scored regions and 6.4 to 38 times slower, on the L4 and the CPU respectively, and it has no efficient form on the TPU. The reduced-precision options change cost and error by less. Float16 throughout saves 9% of the L4 cost for 1.0 dB of error. Single-pass TPU products save 25 to 38% of the TPU cost for about 12 dB, with phase errors of up to 173° beside bright returns (Section 5.2).

Two residuals remain unexplained: the growth of the polar-format error toward the scene edges and the -31.5 to -31.9 dB floor of ISCE3’s off-center Panama patches. On the Capella collections the ISCE3 comparison turns on the receive model adopted for the reference; with its own model ISCE3 has the lower error on three of the five. The cost comparison covers on-demand prices on one GPU and two TPU generations, with the L4 on the smallest host offered and the Capella L4 runs on a larger one (Table 5). H100 and TPU7x instances could not be obtained under the project’s quota (Appendix H).

Acknowledgments. Portions of the code and of this text were drafted with a large language model. The authors designed the experiments, ran and checked the measurements, and are responsible for

the content.

Code and data. FastSAR is available at <https://github.com/saulpingerman/FastSAR> and from the Python Package Index (`pip install fastsar`). The measurements were made with release 0.1.0 and the release candidates named in Appendix H; release 0.1.1, current at the time of writing, contains those candidates' changes together with fixes to the file readers that the measurements do not exercise. The cloud run scripts, the measurement records behind every table and the source of this paper are at <https://github.com/saulpingerman/sar-accel-study>. The radar data are from the Umbra [26] and Capella [27] open data programs and the optical images from Copernicus Sentinel-2 [28]; none is redistributed, and the scripts fetch them from their public sources.

References

- [1] C. V. Jakowatz, Jr., D. E. Wahl, P. H. Eichel, D. C. Ghiglia, and P. A. Thompson, *Spotlight-Mode Synthetic Aperture Radar: A Signal Processing Approach*. Boston, MA: Kluwer, 1996.
- [2] L. A. Gorham and L. J. Moore, "SAR image formation toolbox for MATLAB," in *Proc. SPIE 7699, Algorithms for Synthetic Aperture Radar Imagery XVII*, 2010, p. 769906.
- [3] B. Cordes and M. Leiser, "Parallel backprojection: A case study in high-performance reconfigurable computing," *EURASIP J. Embedded Syst.*, vol. 2009, art. 727965, 2009.
- [4] J. Park, P. T. P. Tang, M. Smelyanskiy, D. Kim, and T. Benson, "Efficient backprojection-based synthetic aperture radar computation with many-core processors," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '12)*, Salt Lake City, UT, 2012.
- [5] A. R. Fasih and T. D. R. Hartley, "GPU-accelerated synthetic aperture radar backprojection in CUDA," in *Proc. IEEE Radar Conf.*, Washington, DC, 2010, pp. 1408–1413.
- [6] T. M. Benson, D. P. Campbell, and D. A. Cook, "Gigapixel spotlight synthetic aperture radar backprojection using clusters of GPUs and CUDA," in *Proc. IEEE Radar Conf.*, 2012, pp. 853–858.
- [7] A. Capozzoli, C. Curcio, and A. Liseno, "Fast GPU-based interpolation for SAR backprojection," *Progress In Electromagnetics Research*, vol. 133, pp. 259–283, 2013.
- [8] Y. Xu, Z. Zhang, L. Chen, and L. Yang, "The adaptive streaming SAR back-projection algorithm based on half-precision in GPU," *Electronics*, vol. 11, no. 18, art. 2807, 2022.
- [9] J. W. McCorkle and M. Rofheart, "An order $N^2 \log(N)$ backprojector algorithm for focusing wide-angle wide-bandwidth arbitrary-motion synthetic aperture radar," in *Proc. SPIE 2747, Radar Sensor Technology*, 1996, pp. 25–36.
- [10] L. M. H. Ulander, H. Hellsten, and G. Stenström, "Synthetic-aperture radar processing using fast factorized back-projection," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 39, no. 3, pp. 760–776, 2003.
- [11] O. Frey, C. Magnard, M. Rüegg, and E. Meier, "Focusing of airborne synthetic aperture radar data from highly nonlinear flight tracks," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 6, pp. 1844–1858, 2009.

- [12] O. Ponce, P. Prats, M. Rodriguez-Cassola, R. Scheiber, and A. Reigber, “Processing of circular SAR trajectories with fast factorized back-projection,” in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, 2011, pp. 3692–3695.
- [13] W. G. Carrara, R. S. Goodman, and R. M. Majewski, *Spotlight Synthetic Aperture Radar: Signal Processing Algorithms*. Norwood, MA: Artech House, 1995.
- [14] C. V. Jakowatz, Jr. and N. Doren, “Comparison of polar formatting and back-projection algorithms for spotlight-mode SAR image formation,” in *Proc. SPIE 6237, Algorithms for Synthetic Aperture Radar Imagery XIII*, 2006, art. 62370H.
- [15] A. W. Doerry, “Wavefront curvature limitations and compensation to polar format processing for synthetic aperture radar images,” Sandia National Laboratories, Albuquerque, NM, Rep. SAND2007-0046, 2007.
- [16] L. A. Gorham and B. D. Rigling, “Scene size limits for polar format algorithm,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 52, no. 1, pp. 73–84, 2016.
- [17] D. Macdonald, “RITSAR: Synthetic aperture radar simulation and processing toolbox for Python,” 2015. [Online]. Available: <https://github.com/dm6718/RITSAR>
- [18] GEOINT, “GRDL: GEOINT Rapid Development Library,” 2026, commit de3338a. [Online]. Available: <https://github.com/GEOINT/grdl>
- [19] California Institute of Technology, “ISCE3: InSAR Scientific Computing Environment, version 3,” releases 0.25.17 and, for the sliding spotlight, 0.25.21, accessed October 2026. [Online]. Available: <https://github.com/isce-framework/isce3>
- [20] H. Forstén, “torchbp: Fast SAR backprojection with PyTorch,” 2025, commit b371884. [Online]. Available: <https://github.com/Tt1/torchbp>
- [21] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, 2017, pp. 1–12.
- [22] N. P. Jouppi *et al.*, “TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings,” in *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*, Orlando, FL, 2023.
- [23] T. Lu, Y.-F. Chen, B. Hechtman, T. Wang, and J. Anderson, “Large-scale discrete Fourier transform on TPUs,” arXiv:2002.03260, 2020.
- [24] T. Lu, T. Marin, Y. Zhuo, Y.-F. Chen, and C. Ma, “Nonuniform fast Fourier transform on TPUs,” in *Proc. IEEE 18th Int. Symp. Biomedical Imaging (ISBI)*, 2021.
- [25] A. G. M. Lewis *et al.*, “Large-scale distributed linear algebra with tensor processing units,” *Proc. Natl. Acad. Sci. USA*, vol. 119, no. 33, 2022.
- [26] Umbra Lab, Inc., “Umbra Open Data Program,” 2023. [Online]. Available: <https://umbra.space/open-data>
- [27] Capella Space, “Capella Space Synthetic Aperture Radar Open Dataset,” 2023. [Online]. Available: https://registry.opendata.aws/capella_opendata/

- [28] European Space Agency, “Copernicus Sentinel-2 Level-2A surface reflectance, tiles S2B_17PPK_20230723 and S2A_17PPK_20240223,” contains modified Copernicus Sentinel data 2023 and 2024, accessed October 2026 through the Earth Search STAC catalog. [Online]. Available: <https://registry.opendata.aws/sentinel-2-l2a-cogs/>
- [29] T. T. Taylor, “Design of line-source antennas for narrow beamwidth and low side lobes,” *IRE Trans. Antennas Propag.*, vol. 3, no. 1, pp. 16–28, 1955.
- [30] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.
- [31] S. Xiao, D. C. Munson, Jr., S. Basu, and Y. Bresler, “An $N^2 \log N$ back-projection algorithm for SAR image formation,” in *Conf. Rec. 34th Asilomar Conf. Signals, Systems and Computers*, 2000, pp. 3–7.
- [32] J. F. Kaiser, “Nonrecursive digital filter design using the I_0 -sinh window function,” in *Proc. IEEE Int. Symp. Circuits and Systems*, 1974, pp. 20–23.
- [33] S. Wang and P. Kanwar, “BFloat16: The secret to high performance on Cloud TPUs,” Google Cloud Blog, Aug. 2019.
- [34] J. Bradbury *et al.*, “JAX: Composable transformations of Python+NumPy programs,” 2018. [Online]. Available: <https://github.com/google/jax>
- [35] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: SIAM, 2002.
- [36] Google Cloud, “Cloud TPU documentation,” accessed October 2026. [Online]. Available: <https://cloud.google.com/tpu/docs>
- [37] Google Cloud, “TPU v6e,” Cloud TPU documentation, accessed October 2026. [Online]. Available: <https://cloud.google.com/tpu/docs/v6e>
- [38] Google Cloud, “Introducing Trillium, sixth-generation TPUs,” May 2024. [Online]. Available: <https://cloud.google.com/blog/products/compute/introducing-trillium-6th-gen-tpus>
- [39] JAX developers, “Pallas: a JAX kernel language,” JAX documentation, accessed October 2026. [Online]. Available: <https://docs.jax.dev/en/latest/pallas/index.html>
- [40] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on Machine Learning Systems (LearningSys) at NIPS*, 2017.
- [41] NVIDIA Corporation, “CUDA C++ Programming Guide,” Section 5.4.1, Arithmetic instructions, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [42] L. R. Rabiner, R. W. Schafer, and C. M. Rader, “The chirp z-transform algorithm,” *IEEE Trans. Audio Electroacoust.*, vol. 17, no. 2, pp. 86–92, 1969.
- [43] Introl, “Google TPU architecture: 7 generations explained,” 2025. [Online]. Available: <https://introl.com/blog/google-tpu-architecture-complete-guide-7-generations>

- [44] GPU Advisor, “TPU v6e (Trillium) specs,” 2025. [Online]. Available: <https://gpuadvisor.com/gpu/tpu-v6e>
- [45] R. K. Raney, “Synthetic aperture imaging radar and moving targets,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-7, no. 3, pp. 499–505, 1971.

A Algorithms

The phase history is motion compensated to the scene reference point. For pulse p with antenna position $\mathbf{a}_p$ and frequency sample f_k ,

$$S_{p,k} = \sum_n \sigma_n \exp\left(-j \frac{4\pi f_k}{c} \Delta R_p(\mathbf{x}_n)\right), \quad \Delta R_p(\mathbf{x}) = \|\mathbf{x} - \mathbf{a}_p\| - \|\mathbf{a}_p\|, \quad (1)$$

where σ_n and $\mathbf{x}_n$ are the complex amplitude and position of scatterer n and c is the speed of light.

The CPHD files give the transmit and receive positions per pulse; their midpoint was used as $\mathbf{a}_p$ (ISCE3 [19] instead places the receiver at the transmit position advanced by the velocity times the delay, the receive model of Section 5.4). They also give the start frequency and sample spacing per pulse; the mean values were used, which differ from the per-pulse values by at most 0.02 of a sample spacing.

A Taylor weighting ($\bar{n} = 4$, -35 dB sidelobes) [29] was applied along both axes in every algorithm. The image grid is the vendor’s: pixel (i, j) lies at $(i - n_x/2) s_x \mathbf{e}_1 + (j - n_y/2) s_y \mathbf{e}_2$ from the reference point. Here $\mathbf{e}_1$ and $\mathbf{e}_2$ are the column and row unit vectors of the vendor’s Sensor Independent Complex Data (SICD) product and s_x , s_y its sample spacings.

Exact backprojection. In the reference each pulse is range compressed by a zero-padded inverse FFT with 16 times oversampling. Each pixel is the sum over pulses of the linearly interpolated profile at ΔR_p times the carrier phase $\exp(j4\pi f_c \Delta R_p/c)$ [2]. The range difference is evaluated in the cancellation-free form

$$\Delta R_p(\mathbf{x}) = \frac{q}{1 + \sqrt{1 + q/r_p}}, \quad q = \frac{\|\mathbf{x}\|^2}{r_p} - 2 \hat{\mathbf{u}}_p \cdot \mathbf{x}, \quad (2)$$

with $r_p = \|\mathbf{a}_p\|$ and $\hat{\mathbf{u}}_p = \mathbf{a}_p/r_p$. The reference implementation evaluates everything in float64 with Numba [30] on the CPU.

FastSAR’s exact backprojection (**ExactFormer**, with C++ and CUDA kernels) evaluates the range, the unit vector and the reciprocal range of each tile center in float64 once per pulse. Each pixel’s range is then the center’s plus a third-order expansion in its offset $\mathbf{d}$ from the center, $\mathbf{d} \cdot \hat{\mathbf{u}} + D/(2r) - (\mathbf{d} \cdot \hat{\mathbf{u}})D/(2r^2)$ with $D = \|\mathbf{d}\|^2 - (\mathbf{d} \cdot \hat{\mathbf{u}})^2$, evaluated in float32. The tile, 32 by 32 pixels at orbital range, is reduced where the predicted error of the next term would exceed 3×10^{-4} rad. Each pulse is range compressed by a zero-padded FFT, and only the bins its tiles can reach are kept. The profiles are read with cubic Lagrange interpolation (oversampling 8 from release 0.1.1, 4 in release 0.1.0) or linear interpolation (oversampling 8, or 16 on request). On the L4 the Panama image, 1.6 trillion interpolated reads, takes 24.1 s, or 68 billion reads per second.

Factorized backprojection. The image is divided recursively into tiles, as in the quadtree [9, 31] and fast factorized [10] backprojection algorithms. At each level the phase history of a parent tile is multiplied by $\exp\{j4\pi f_k(\|\mathbf{c} - \mathbf{a}_p\| - \|\mathbf{c}' - \mathbf{a}_p\|)/c\}$ to move its reference point from the parent

Table 6: Floating-point type of each step in the timed configurations. Geometry means tile centers, ranges and phase-ramp coefficients; ramps means the sines and cosines themselves; products means the operands of the matrix products, filters or sums, always accumulated in float32 or better.

Device	Algorithm, label	Data in memory	Geometry	Ramps	Products
L4, CPU	Exact BP, cubic or linear interpolation	float32 profiles	float64 per tile, float32 within the tile	float32	float32 sums
L4	Factorized BP, float32, CUDA kernels	float32	float64	float32	float32 fused multiply-adds (levels and final)
L4	Factorized BP, float16 throughout, CUDA kernels	float16	float64	float32	float16 data into float32 fused multiply-adds in the levels; float16 operands on the tensor cores in the final stage
TPU	Factorized BP, three-pass products, fused kernels	float32	float64 (2 levels), float32	float32	3 bfloat16 passes (levels and final)
TPU	Factorized BP, single-pass products, fused kernels	float32	float64 (2 levels), float32	float32	bfloat16 operands, 1 pass (levels and final)
CPU	Factorized BP, float32, C++ kernels	float32	float64	float32	float32 fused multiply-adds (levels and final)
CPU	Factorized BP, float32, JAX program	float32	float64 (2 levels), float32	float32	float32
all	Polar format, float32	float32	float64	float32	float32 sums over gathered samples; float32 FFTs

center $\mathbf{c}'$ to a child center $\mathbf{c}$. The child’s signal is then band limited in frequency and in pulse index in proportion to the child’s size, and both axes are low-pass filtered and decimated by a Kaiser-windowed sinc [32] with 70 dB design attenuation.

At the last level the range to each pixel is linearized about the tile center and the kernel separates in the two image coordinates. The tile image is then a product of two matrices followed by a per-pixel quadratic phase correction, and no stage requires an indexed read.

Rectangular images of arbitrary size are handled: the side of each level divides the padded image by factors chosen from 8, 7, 6, 5, 4, 3 and 2. The decimation factors follow from the range and Doppler extents of a child tile. For the Panama image the three levels divide the azimuth side by 8, 8 and 6 and the range side by 8, 7 and 5. The first level decimates the 14,399 samples by 6 and the 15,186 pulses by 4. The final tiles are 32 pixels on a side on all three collections, the default size, which the predicted error supports for each collection geometry.

In the JAX program and the TPU kernels the first two levels compute their phase ramps from float64 geometry on the host and the remaining level from float32 geometry on the device. The CUDA and C++ kernels use float64 geometry at every level. During development a float32 second level was found to add an error that grew with the size of the scene. The ramps themselves are evaluated on the device in float32. The decimation filters are applied as dense matrix products on the TPU, as strided convolutions in the JAX program on the CPU, and as explicit fused multiply-adds in the CUDA and C++ kernels.

B Floating-point formats and the arithmetic of each configuration

Table 6 gives the floating-point type of each step of every timed configuration; a development configuration in which the phase ramps were also computed in bfloat16 produced an unusable image, and release 0.1.0 does not offer it.

Float16 (IEEE binary16, Table 7) overflows above 65,504, so the phase history is scaled to unit peak before conversion. On the TPU every matrix product is formed from bfloat16 [33] operands. A float32 operand is either rounded once (one pass) or split into a bfloat16 head and tail with the cross terms accumulated (three passes); the three-pass product keeps the first 16 significant bits of each operand. A split into three parts (six passes, used by the JAX program) recovers float32 accuracy [34]. The L4’s tensor cores accept float16 operands, which the final stage of float16 throughout uses. The CPU kernels use no reduced-precision arithmetic; the AVX-512 bfloat16 instructions of the processor were not evaluated.

Table 7: Floating-point formats. Significant bits include the implicit bit; the spacing is the distance between adjacent representable values near 1.

Format	Bits	Significant bits	Exponent bits	Spacing near 1
float64	64	53	11	2.2×10^{-16}
float32	32	24	8	1.2×10^{-7}
float16 (IEEE binary16)	16	11	5	9.8×10^{-4}
bfloat16	16	8	8	7.8×10^{-3}

The measured image errors are consistent with the operand resolutions of Table 7. Rounding each operand of the factorized products to bfloat16 adds noise about 48 dB below the operand [35]. Once the error of the three-pass image is removed, the measured error of the single-pass TPU image agrees to within about 1 dB with the rounding noise of the two operands accumulated through three levels. The L4’s float16 operands have 18 dB finer resolution than bfloat16, and their rounding adds 1.0 dB to the error of the float32 CUDA build, whose products are all float32.

C Exact backprojection on the TPU

FastSAR has no TPU kernel for exact backprojection, because the TPU has no fast path for its per-pixel lookup; on a TPU **ExactFormer** runs in pure JAX (linear interpolation of profiles oversampled 32 times, the accuracy of the cubic default). Timed at the 0.1.1 release candidate **rel-rc6-20261009** on 512 by 512 pixel grids centered on the Panama lock region, with all pulses, it performed 3.0×10^7 interpolated reads per second on the v5e and 9.2×10^6 on the v6e. At those rates the 1.6 trillion reads of the Panama image take about 15 h (about \$18 per image at the instance price) and 49 h (about \$133), compared with 24 s on the L4. The grids agreed with the reference to -65.9 and -66.0 dB. On a 256 by 256 grid the same program ran at 2.7×10^7 reads per second on the v6e; the slower of the two measurements is used above, and the 49 h figure is therefore an upper estimate.

A TPU core has one or more matrix units, which multiply dense bfloat16 tiles at the chip’s rated rate, and a vector unit. In microbenchmarks, data-dependent one-dimensional gathers on the vector unit ran at about 10^8 reads per second and the scattered two-dimensional gathers of the polar-format resampling at about 4×10^7 (Appendix E). Both rates are two orders of magnitude below the matrix units’ rate per element.

Google’s Cloud TPU documentation [36] describes no texture or cache path for scattered reads. The sparse cores of the v6e, intended for embedding lookups, are documented near 10^9 rows per second per chip [37], which would still need about 30 min for the Panama image. A one-hot matrix product would move the lookup onto the matrix units, but its matrix has one nonzero in several thousand entries, and the dense product would exceed the work of the factorized algorithm by a comparable factor. The closest published work computes FFTs and nonuniform FFTs, the latter for MRI reconstruction, as matrix products on the TPU [23, 24].

D Kernels for the factorized algorithm

The factorized rows of Table 4 are formed by kernels written for each device after a stage-by-stage profile of the code that XLA, JAX’s compiler, generated from the JAX program. On the device, the JAX program took 4.6, 9.3 and 13.7 s for the float32-class Panama image on the v6e, v5e and

Table 8: The builds of the factorized algorithm on Panama, two precisions per accelerator and float32 on the CPU: 16-bit means single-pass products on the TPUs and, on the L4, float16 products (JAX program), the float32 build with the float16 tensor-core final stage (CUDA builds four to six) or float16 storage throughout (build seven); float32-class means three-pass products on the TPUs and float32 on the L4. The first row of each device is the device time of the JAX program per image (s), measured in the development session of the first kernel build (on the CPU, which has no transfers, the JAX program of the released library); the rows between give the change each development build made to the device time of the build before it (s; negative is faster). Each change is relative to the nearest row above that is not marked rejected or instead. The last row is the time of the released library from host memory to host memory (Table 4), which includes the transfers and the per-collection host work and is not directly comparable with the device time of the last build (on the CPU, which has no transfers, the release time is the shorter). On the CPU the changes add up from the first C++ build (51.8 s, Appendix D), the step from the JAX program to it being omitted.

Device	Build	16-bit (s)	float32-class (s)
TPU v6e	XLA: ramps materialized, dense decimation products, XLA final stage	3.82	4.60
	Fused level-0 kernel, one child per load	+0.69	+0.65
	Fused level kernels, children share a load, XLA final stage	-0.41	-0.50
	Same, tile level batched over parents	-1.04	-1.03
	Plus fused final stage, per-sample sines	-0.04	-0.02
	Plus fused final stage, ramps by recurrence	-1.02	-0.95
	Plus clipped windows, no halo at levels 1 and 2, precomputed fine tables	-0.11	-0.13
	Fine tables by doubling instead (rejected)	+2.16	+2.16
	Coarse tables precomputed too, pulse decimation fused into the kernel at levels 1 and 2	-0.21	-0.26
	Four tiles per step in the final stage instead (3% faster on the v6e at three-pass precision only; not kept)	0.00	-0.07
	Coarse-table sines over the used lanes only, padded afterwards (rejected)	+0.10	+0.10
	Level kernels of the row above, one-tile recurrence final stage (last development build)	0.00	0.00
	Release 0.1.0, host memory to host memory	2.58	3.12
TPU v5e	XLA: ramps materialized, dense decimation products, XLA final stage	6.49	9.29
	Fused level-0 kernel, one child per load	+1.23	+0.82
	Fused level kernels, children share a load, XLA final stage	-0.94	-1.55
	Same, tile level batched over parents	-1.75	-1.63
	Plus fused final stage, per-sample sines	+0.10	+0.27
	Plus fused final stage, ramps by recurrence	-1.71	-1.62
	Plus clipped windows, no halo at levels 1 and 2, precomputed fine tables	-0.23	-0.49
	Fine tables by doubling instead (rejected)	+3.68	+3.67
	Coarse tables precomputed too, pulse decimation fused into the kernel at levels 1 and 2	-0.43	-0.52
	Four tiles per step in the final stage instead (3% faster on the v6e at three-pass precision only; not kept)	+0.10	+0.10
	Coarse-table sines over the used lanes only, padded afterwards (rejected)	+0.23	+0.23
	Level kernels of the row above, one-tile recurrence final stage (last development build)	0.00	0.00
	Release 0.1.0, host memory to host memory	3.63	5.20
L4	XLA: ramps materialized, cuDNN convolutions, XLA final stage	11.16	13.69
	Triton level-0 kernel (Pallas), 16-column windows (not kept; the CUDA rows below are measured from it)	-0.43	+5.03
	CUDA: shared-memory FIR levels, one pixel per thread final	-	-13.10
	Final stage 2 by 2 pixels per thread	-	-1.01
	Pulse-major filter threads (conflict-free reads, scattered loads and stores), tiled pulse filter, 4 by 2 final	-	+0.84
	Coalesced window loads, children as in-place phase steps; float16 tensor-core final	-6.98	-1.05
	Rotation as a geometric sequence per thread; interleaved complex tables in the final	-0.20	-0.42
	Output tiles staged in shared memory for coalesced stores (last float32 development build)	-0.39	-0.39
	Float16 phase history, intermediates and final-stage operands, float32 accumulation in the filters (float16 build, as released)	-0.26	-0.07
	Release 0.1.0, host memory to host memory	3.48	3.78
CPU	JAX program: conv. filters, direct ramps, one process	-	224.45
	C++ with OpenMP: polyphase filter vectorized along the output columns (stride- D gathers)	-	-
	Children in the vector lanes (no gathers), input rows read once in the pulse filter, final tables by two recurrences	-	-32.02
	Named accumulators, complex products as FMA pairs, 512-bit vectors, reused intermediates	-	-6.58
	Final-stage tables built and consumed three pulses at a time (second-level cache); released build	-	-0.88
	Release 0.1.0, host memory to host memory	-	11.31

Table 9: Each stage of the profiled development builds (on the TPUs and the CPU the last build of each device in Table 8, on the L4 the sixth, float32, build) against the lower bound set by the unit that binds it, Panama image, single-pass or float32 arithmetic. The bounds divide the inherent work of the stage by rates measured on the same device with microbenchmarks in the same framework (vector unit on the rotation mix of the kernel with data resident on chip, high-bandwidth memory copy, matrix units at the product shapes of the kernel, sines in XLA; on the L4 the inner mixes of the kernels on resident shared memory and the memory copy rate). Ratio is measured time over bound; the last column divides the measured time by the sum of all the bounds of the stage, the time it would take if none of its units overlapped. Stage times are measured on the device, excluding transfers.

Device	Stage	Binding unit	Bound (s)	Measured (s)	Ratio	To the sum
TPU v6e	level 0	vector unit	0.15	0.22	1.4	0.60
	level 1	vector unit	0.30	0.30	1.0	0.52
	level 2	sines (XLA)	0.25	0.61	2.5	0.88
	final	matrix units	0.13	0.29	2.2	–
TPU v5e	level 0	matrix units	0.33	0.50	1.5	0.72
	level 1	matrix units	0.38	0.60	1.6	0.62
	level 2	sines (XLA)	0.40	0.88	2.2	0.78
	final	matrix units	0.25	0.44	1.7	–
L4	levels (rotation and frequency filter)	memory	0.73	1.36	1.9	1.08
	pulse filter	memory	0.41	0.46	1.1	1.11
	final (float32)	arithmetic	0.45	0.98	2.2	2.07
CPU	levels (rotation and frequency filter)	arithmetic	4.19	4.87	1.2	0.79
	pulse filter	memory	2.06	2.17	1.1	0.84
	final (float32)	arithmetic	2.67	3.76	1.4	1.35

L4, and the last development builds 2.4, 4.6 and 3.5 s (the L4 figure is the seventh session’s float32 build; its float16-storage build took 2.9 s). On the CPU release 0.1.0 forms the image in 224 s with the JAX program and 11.3 s with the C++ kernels. The float32 images of the CUDA and C++ kernels are closer to the reference than that of the JAX program, a difference consistent with their float64 geometry at every level.

In the terms of Appendix A, a level splits each parent tile’s phase history into those of its children, the tiles of the next level. Level 0 splits the whole collection into the first-level tiles; levels 1 and 2, which run inside one first-level tile per device call, are called the tile levels below. In the JAX program and the TPU kernels only level 2 takes its geometry from the device; the CUDA and C++ kernels compute it in float64 on the host. The plan and filters are the same in every build, and each build’s image was checked against the dense image at the same precision on a simulated scene before it was timed. In Table 8, 16-bit means single-pass products on the TPUs and, on the L4, the float16 variant of each build that the caption lists; float32-class means float32 on the L4 and the CPU and three-pass products on the TPUs.

Diagnosis. In development, on the v6e, one first-level tile of the Panama image took 51 ms in the JAX program with single-pass products: 8.3 ms in level 0, 13.3 in level 1, 7.9 in level 2 and 21.4 in the final stage. The three levels are bound by memory traffic. XLA writes the rotated phase history of every child (24 bytes per sample, read back by the decimation product) at about 580 GB/s, a third of the v6e’s nominal 1,640 GB/s [38] (1,391 GB/s measured, below). The children of the three levels total 63 billion samples per image.

The final stage materializes the two ramp matrices of every 32 by 32 tile ($2T$ by $P_f Q_f$ each, with T , P_f and Q_f the tile side and the pulses and frequency samples per tile, defined below) through five or six element-wise passes before the one product that uses them. Its direct form evaluates 400,000 sines and cosines per tile. The same distribution of time is found on the L4, at the lower bandwidth of its convolution path.

TPU kernels (Pallas). Two Pallas kernels replace the XLA stages. The level kernel takes a block of pulses and a window of columns of a parent into the core’s vector memory and applies the phase ramps of up to eight children in turn. Each ramp is the product of a coarse table, one angle per 128-column chunk, and a fine table, one angle per column within the chunk. Both are precomputed once per pulse and child by XLA, and the kernel evaluates no sines. The Pallas documentation [39] lists sines among its most expensive operations, and an earlier build that evaluated the coarse table in the kernel spent a third of its time on them.

The rotated window is multiplied by the banded block of the decimation matrix on the matrix units, in one or three bfloat16 passes. At the two tile levels the pulse-axis decimation is accumulated on the matrix units across the pulse blocks in the same kernel, and neither the rotated window nor the frequency-decimated child reaches high-bandwidth memory. At level 0 the decimated child is too large for vector memory, and the pulse product is a separate dense product.

Children of one parent share each load, and the tile levels batch the parents of a tile into one launch. First-level tiles are formed in groups, and the phase history is read once per group. The development builds of Table 8 used groups of 16; release 0.1.0 uses groups of 4 and also applies the level-0 pulse decimation as a banded product. Where one block of outputs covers a whole row (the tile levels), the window is clipped to the data and read once; at level 0 consecutive output blocks overlap by the filter length.

The final kernel forms the $2T \times 2T$ product of Appendix A on the matrix units, where $T = 32$ is the tile side in pixels and $P_f \times Q_f$ the pulses by frequency samples per tile. Both ramps are generated by a complex recurrence along the frequency index q . For a pixel offset d along an image axis and the component u_p of the unit vector to the antenna along that axis, the ramp is $\exp(-j2\pi du_p(a_0 + a_1 q))$. Here a_0 and a_1 are the cycles per meter at the first frequency sample and per sample step. This equals XW^q with $X = \exp(-j2\pi du_p a_0)$ and $W = \exp(-j2\pi du_p a_1)$, and one tile needs four tables of sines and cosines of size $T \times P_f$ instead of one sine per sample.

A first version of this kernel, with one sine per sample, was no faster than the XLA final stage (Table 8); the recurrence eliminated the sine evaluations, and the stage was no longer bound by the transcendental unit. Other variants tried in development were not kept. Building the level kernel’s fine tables in XLA by successive doubling nearly doubled the image time, because each intermediate table of fewer than 128 columns occupies full 128-lane tiles in TPU memory. Evaluating the coarse-table sines over the used lanes alone and padding afterwards cost 4 to 8%, since XLA holds the table 128 lanes wide either way. Stacking four tiles into one 256 by 256 product in the final stage gained 3% on the v6e at three-pass precision and nothing otherwise, and lost 2 to 4% on the v5e, which has 128 by 128 arrays; these differences are within the run-to-run spread of Appendix H.

Bounds. The bounds of Table 9 are for the profiled builds of Table 8, timed on the device; the released library’s times from host memory to host memory are those of Table 4.

The unit rates were measured on each device with microbenchmarks in the same framework (`ubench_tpu.py` and `ubench_gpu.py` in the repository). On the v6e, high-bandwidth memory copies run at 1,391 GB/s. The vector unit executes the level kernel’s rotation mix on data resident in vector memory at 117 billion samples per second and a plain copy through vector memory at 155

billion. The rotation therefore runs within a factor of 1.3 of the vector load and store path. XLA evaluates 57 billion sine and cosine pairs per second, and the matrix units reach 457 TFLOP/s on a large bfloat16 product and 42 TFLOP/s at the final stage’s 64-row shape.

Against these rates, on the v6e, level 1 runs at the vector unit’s rate (ratio 1.0) and level 0 at 1.4 times its vector bound, with the unfused pulse product passing through memory. Level 2 runs at 2.5 times its largest bound but at 0.9 of the sum of the terms that run in sequence: the rotation (0.24 s per image), the sines XLA evaluates for its tables (0.25 s) and the traffic of those tables (0.17 s). The level’s rows are 352 samples long, so each child and pulse needs 256 sines for 512 rotated samples. The final stage is 2.2 times its matrix bound; the difference is attributed to the ramp recurrence on the vector unit, which the microbenchmarks did not isolate.

On the v5e the matrix units are about three times slower than the v6e’s on a large product, against a vector unit only 12% slower, and they become the binding unit of levels 0 and 1; each v5e stage lies between its largest bound and the sum of its bounds (1.5 to 2.2 times the former, 0.6 to 0.8 of the latter), so its units overlap only partly.

The largest remaining inefficiency identified within the factorization is the level-2 table generation, which angle-addition tables could reduce by about a factor of eight, about 0.2 s of the 1.7 s device time of the v6e image; the other TPU stages are at most 2.2 times their largest single bound and, where a sum is defined, below the sum of their bounds.

L4 kernels (CUDA). On the L4 the two kernels are written in CUDA and compiled through CuPy [40]. The level kernel loads the window of 32 pulses into shared memory with consecutive threads on consecutive columns, so that global loads are coalesced. It then rotates the window in place for each child, generating the rotation of each thread’s samples as a geometric sequence. Each thread then computes two adjacent outputs of one pulse with the 44-tap decimation filter as fused multiply-adds. For this phase consecutive threads are consecutive pulses, so no two threads of a group read the same shared-memory bank. The outputs are staged through shared memory and written row by row. The pulse-axis filter is tiled the same way.

The final kernel builds the two ramp tables of a tile in shared memory by the same recurrence, and each thread accumulates a 4 by 2 block of pixels. In the float16 tensor-core builds the tables are converted to float16 and the $2T \times 2T$ product runs on the tensor cores with float32 accumulation. A Triton version of the level kernel through Pallas, with the 16-column windows its shared-memory limit allows, gained 1 to 4% with float16 products and was slower in float32 (development device timings, within the run-to-run spread of Appendix H); the library uses the CUDA builds.

Seven CUDA builds were made and five profiled (Table 8; the profiles are in the repository). The profile of an early build placed 1.5 s per image in the level kernel, 1.3 in the final stage and 0.5 in the pulse filter. A pulse-major thread mapping removed the bank conflicts from the filter reads but scattered that kernel’s global loads and stores, and its time doubled. The following three builds added coalesced loads through shared memory, the geometric-sequence rotation and staged stores, and brought the kernel to 1.9, 1.7 and 1.4 s. The float32 final stage appeared bound by the latency of its table-building recurrence and by one shared-memory word per fused multiply-add; with a 4 by 2 block per thread and interleaved complex tables it fell from 1.3 to 1.0 s. The tensor-core version was 0.43 s faster per image on the device than the float32 one in the sixth build. The seventh build stores the phase history, every intermediate and the final-stage operands in float16 and accumulates in float32 inside the filters, which halves the memory traffic that bounds the level kernel and the pulse filter. It is the float16 configuration of Table 4; release 0.1.0 forms the image with it in 3.5 s, against 3.8 s for the float32 build, with 1.0 dB more error.

The L4’s measured rates are a memory copy at 246 GB/s, the level kernel’s inner mix on resident

shared memory at 20.2 billion filter outputs per second, the final kernel’s inner mix at 6.1 trillion fused multiply-adds per second (41% of the chip’s peak), and coalesced stores at 252 GB/s, compared with 69 GB/s for strided stores.

Against these, the pulse filter runs at its memory bound and the level kernel at 1.1 times the sum of its arithmetic and memory bounds (1.9 times their maximum; the two overlap only partly at two blocks per multiprocessor). The float32 final stage runs at 2.2 times the bound of its inner mix; the profile places the excess in the table building.

The remaining structural cost of the float32 build is the 172 GB per image that the level kernel writes and the pulse filter reads back; a streaming fusion of the two could remove about half of it, about 0.3 s of the 3.0 s that the profile of the sixth build measures on the device (the three stages of Table 9 account for 2.8 s of it; the development session’s own timing of that build, 3.6 s, includes host work). The float16 build halves that traffic instead. In release 0.1.0 it forms the image 0.3 s faster than the float32 build (Table 4), less than halving a purely memory-bound stage would give. The level kernel’s arithmetic bound is close to its memory bound, and float16 storage leaves the arithmetic bound unchanged, since the L4’s CUDA cores issue float16 and float32 fused multiply-adds at the same rate (128 per clock per multiprocessor at compute capability 8.9 [41]). Further gains from float16 on the L4 would depend on the tensor cores, for which the level filters would have to be rewritten as matrix products, as in the TPU kernel.

CPU kernels (C++). The level filter, the pulse filter and the final stage were written in C++ with OpenMP for the CPU instance, compiled by GCC 13 for AVX-512 with 512-bit vectors preferred, and called from the Python driver through a shared library. The host side (plan, filters, float64 geometry) is the code used for the other devices.

The design that came closest to the arithmetic bound assigns the children of a parent to vector lanes. A sample of the parent row is a scalar broadcast, its rotation one complex multiply per 16 children with a per-lane phase step, and the filter accumulates eight outputs per child vector in registers, so no strided read occurs. The pulse filter reads each input row once per chunk of 16 output rows. The final stage builds its two tables three pulses at a time, by a geometric recurrence over the pulse index and another over the pixel index (four sines per pulse and table). The tables then stay in the second-level cache while the product consumes them.

Four builds were measured (Table 8). The first, a polyphase filter vectorized along the output columns, took 51.8 s per Panama image; its stride- D gathers cost about 12 instructions per useful multiply-add. The second, with children assigned to vector lanes, took 19.8 s. The third build (13.2 s) reused the intermediate buffers, which removed one page fault per 4 KB of fresh allocation inside the parallel regions, and kept the accumulators in registers. The fourth (12.4 s, of which 3.8 s was the final stage) builds and consumes the final-stage tables three pulses at a time. The fourth build is the one released; it ran on 8 threads, one per core, and 16 threads gave the same time.

The instance’s measured rates are a memory copy at 49 GB/s, the filter’s inner loop at 314 billion fused multiply-adds per second on resident data, and the final product’s inner loop at 1.04 trillion fused multiply-adds per second. The last figure is near the 8 cores’ float32 peak of two 512-bit fused multiply-adds per core and cycle at the measured clock. Against these, the level kernel runs at 1.2 times its arithmetic bound (0.8 of the sum of arithmetic and memory) and the pulse filter at 1.05 times its memory bound. The final stage runs at 1.4 times its arithmetic bound, with the excess in table building (Table 9).

E Polar format on an orbital aperture

FastSAR’s polar format consists of three resampling steps and a two-dimensional FFT, each expressed with FFTs, element-wise products or a single matrix product so that the same code runs on all devices. The pulses are first resampled so that the tangent of the look angle in the image plane is uniform in pulse index, which an orbit does not give exactly. This is a 16-tap Kaiser-windowed sinc interpolation along the pulse axis, applied as a sum over 16 gathered rows on every device. A dense 15,186 by 15,186 matrix product, also implemented, needed more than the v5e’s 16 GB with its intermediates. Each pulse is then resampled onto a uniform radial wavenumber grid, and each wavenumber row onto a uniform cross-range grid, by chirp-z transforms [42]. These evaluate the band-limited interpolation on a scaled grid with FFTs and chirps; outputs that would wrap around the band are zeroed.

The frequency samples are weighted by f_c/f_k before resampling. Backprojection’s uniform weighting of the samples corresponds to a density of $1/k$ in the wavenumber plane (the polar Jacobian) that the uniform spectral weighting of polar format omits; omitting this weighting worsened the error by about 1.5 dB.

Because the grazing angle changes along the aperture, the radial band of each pulse projected onto the image plane drifts from pulse to pulse. The raster covers the union of the bands and so keeps the full spectral support of backprojection, a trapezoid, rather than the inscribed rectangle. For these collections the inscribed rectangle would keep about half the bandwidth in the ground plane and nearly all of it in the vendor’s slant plane. The wavenumber grid spacings are set by the native pixel spacings and FFT sizes of 13,824 by 10,800, which include a 300 m guard band on every side of the scene. The range and azimuth profiles are tapered to zero within the guard, because the periodic transforms otherwise wrap echoes from beyond the image into its edges (without the guard the edge error was -5 dB in development).

Distortion. Polar format assumes a planar wavefront, and a scatterer at distance ρ from the scene center is then focused about $\rho^2/2R$ from its position in range. At $R = 744$ km (mid-aperture) this is 4 m at 2.5 km, about 12 pixels. The azimuth displacement is larger and depends on the squint (7.6 m in azimuth and 4.3 m in range at the Panama corners). The planar model writes the range history of a scatterer at $\mathbf{p}$ as $-\hat{\mathbf{u}}_p \cdot \mathbf{q} + c$. A least-squares fit of $\mathbf{q}$ and c over the aperture to the exact history $\|\mathbf{p} - \mathbf{a}_p\| - \|\mathbf{a}_p\|$ gives the position at which polar format focuses the scatterer, $(q_x, q_y + c)$, since a constant range offset is a range shift.

The fit residual is below 1 mrad of phase everywhere in the three scenes, so the image is displaced without defocus, by up to 18 pixels in azimuth and 12 in range at the Panama corners. Measured target positions agreed with the model to 0.2 pixel, and before the final resampling the 5 by 5 coherence with the reference was 0.42. The mapping is fitted by cubic polynomials over the image (residual below 10^{-4} pixel).

Final resampling. The spectrum is placed in a raster of twice the pixel density and transformed without the carrier terms. The result is interpolated at the apparent positions with a six-tap Lanczos kernel along each axis and multiplied by the carriers evaluated at the apparent positions. No separate phase correction is needed; the quadratic phase of about 12 rad at 211 m seen in development in a center crop before the resampling is the carrier of the range displacement. The oversampled raster is processed in blocks: the range transform is done once, and the azimuth transform and the resampling proceed in blocks of 512 output range columns. The polar-format figures in this paper are for the resampled image.

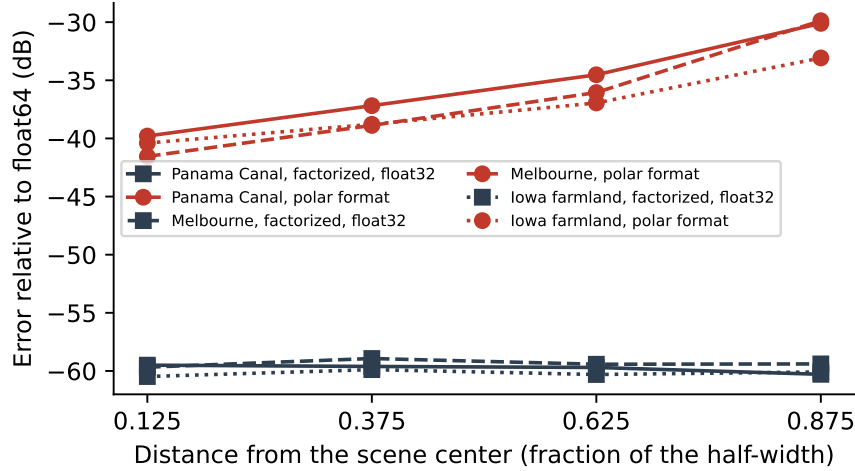


Figure 5: Error relative to the reference in four concentric rings by distance from the scene center, for the polar-format image and the float32 factorized image on the L4, on the three collections (dB; rings are quarters of the half-width from the scene center outward).

Table 10: Panama Canal, polar format on the CPU in development: whole-image error and error by ring (dB) for variations of the final resampling. Rings are the quarters of the half-width from the scene center outward, inner to outer; the selected configuration is the first row.

Variation	Error	Ring 1	Ring 2	Ring 3	Ring 4
6-tap kernel, 2 times oversampled (selected)	-32.1	-39.8	-37.2	-34.5	-30.1
4-tap kernel	-31.6	-37.1	-35.1	-33.5	-29.8
8-tap kernel	-32.2	-40.1	-37.4	-34.7	-30.2
3 times oversampled	-32.2	-40.1	-37.3	-34.7	-30.1
32-tap pulse interpolator, Kaiser 12	-31.7	-37.7	-35.7	-33.8	-29.8
48-tap pulse interpolator, Kaiser 12	-31.2	-36.4	-34.5	-33.1	-29.6

Residual. After the resampling the error is -32.1 dB on Panama (-33.5 dB on Melbourne and -35.4 dB on Iowa) and the mean 5 by 5 coherence 0.9994. On every collection the residual grows from the inner to the outer ring, by 7 to 12 dB, from -40 to -30 dB on Panama, whereas the factorized error shows no dependence on distance from the center (Figure 5). No explanation was found for the residual.

In development, on the full Panama image, the resampling kernel (4, 6 and 8 taps) and the oversampling of the intermediate raster (2 and 3 times) changed the whole-image error by less than 0.6 dB (Table 10). In the outermost ring, where the displacement is 10 to 18 pixels, the change was less than 0.4 dB. Lengthening the pulse interpolator from 16 to 32 or 48 taps, with a narrower Kaiser window, made the error up to 0.9 dB worse. The registration error, measured from the cross-spectrum phase, was 0.001 pixel, and the convergence of the reference is more than 30 dB below the residual. One untested candidate is the Taylor weighting, which backprojection applies per pulse and sample and polar format applies through the resampling onto the rectangular raster; such a difference would grow with distance from the scene center.

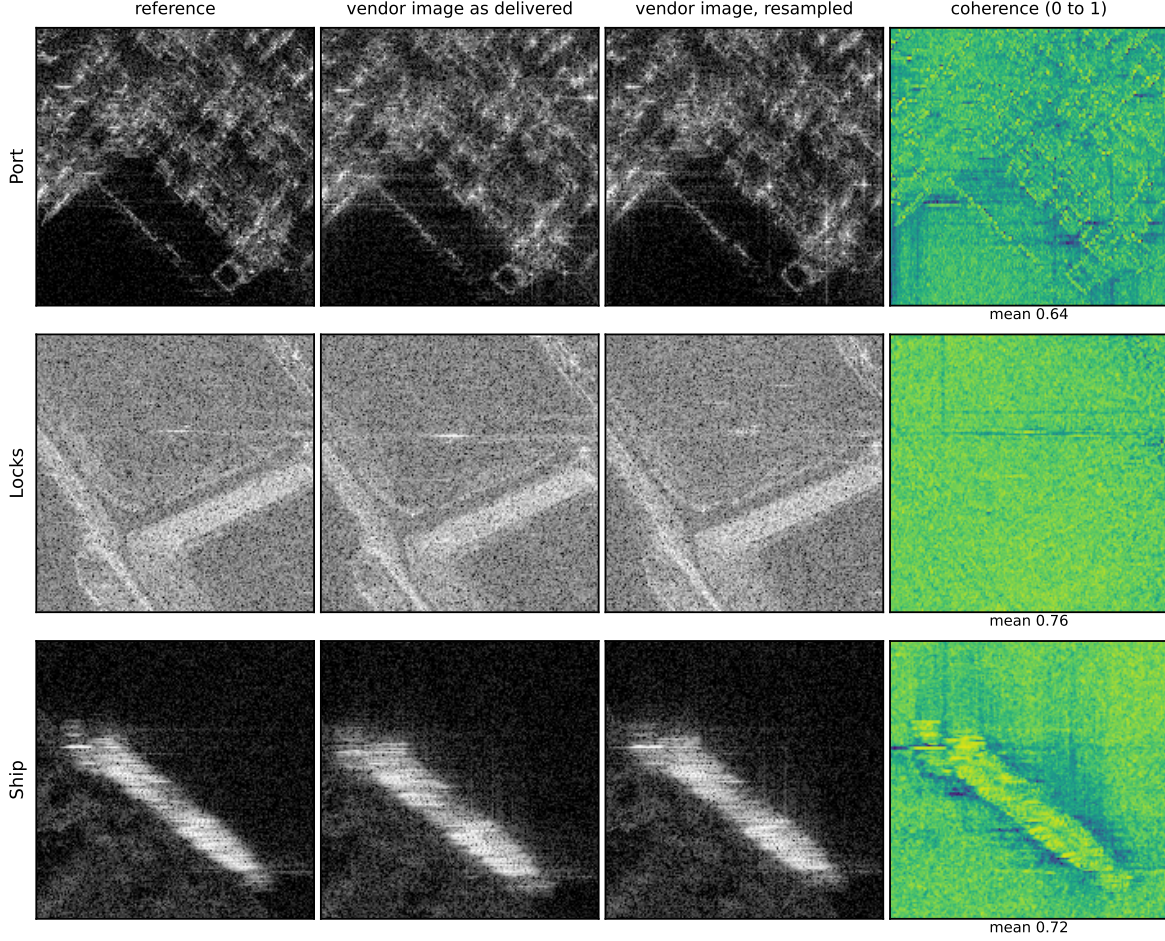


Figure 6: The port, lock and ship regions of Figure 3 (rows, 512 by 512 pixels; the row order differs from that figure). Columns: the reference; the vendor image as delivered, on the same pixels; the vendor image after resampling through the planar-wavefront displacement model of Appendix E; and the 5 by 5 coherence of the resampled vendor image with the reference after local realignment and phase-ramp removal, from 0 to 1, with its mean below each map. Amplitudes are displayed over a 45 dB range.

Time. The final resampling is an indexed read at a computed position, 36 reads per pixel. On Panama the polar-format image takes 3.0 s on the L4 and 11.4 s on the CPU, 0.80 and 1.01 times the factorized float32 time on the same device. On the TPUs it takes 103.4 s (v5e) and 110.2 s (v6e), 20 to 35 times the three-pass factorized time, because its final resampling consists of scattered two-dimensional gathers at the rate given in Appendix C. A separable approximation of the final resampling, expressible as row and column phase ramps, would avoid the scattered gathers. The squinted geometry of these collections is not exactly separable in this way, and the approximation's effect on the TPU's cost was not measured.

Table 11: The reference (float64 exact backprojection) against the vendor’s complex image (SICD) of the same collection, which Umbra forms by polar format. Columns: the number of 512 by 512 windows, centered on a grid with 1024-pixel spacing, with enough contrast for an amplitude cross-correlation; the largest displacement of the vendor image relative to the reference over the windows (azimuth / range, pixels); the largest displacement the planar-wavefront model of Appendix E predicts at the same windows; the root-mean-square difference between measured and predicted displacement; the root-mean-square residual displacement after the vendor image is resampled through the model; and, after that resampling and a local realignment and phase-ramp removal in each window, the 5 by 5 coherence of the two images (mean over all pixels of the scene; median of the window means) and the median over the windows of the correlation of their log amplitudes.

Collection	Windows	Measured (px)	Model (px)	RMS difference (px)	Residual after resampling (px)	Coherence, scene mean	window median	Log-amplitude correlation
Panama Canal	68	13.2 / 9.0	13.2 / 9.1	0.11 / 0.12	0.07 / 0.05	0.46	0.67	0.49
Melbourne	81	7.3 / 7.1	7.4 / 7.2	0.10 / 0.19	0.05 / 0.11	0.67	0.76	0.52
Iowa farmland	64	6.7 / 8.1	6.7 / 8.5	0.07 / 0.34	0.01 / 0.25	0.63	0.80	0.57

F Validation against the vendor’s image

Umbra, the vendor, distributes each collection as a complex image in the SICD format, on the pixel grid that the reference adopts; its metadata states that the image was formed from the same phase history by polar format. This allows the reference to be checked against an independent processor. The integer and subpixel offset between the two images (attributed to the vendor’s reference-pixel convention) was found from the cross-correlation of block-averaged log amplitude and refined on a central window. The linear phase ramp between them, which reflects the different spectral centering of the two images (-0.225 cycles per pixel in range on Panama), was found from the peak of the cross-spectrum of the same window and removed.

The vendor image was compared with the reference in 512 by 512 windows centered on a grid with 1024-pixel spacing, keeping the windows with enough contrast for an amplitude cross-correlation (68 on Panama). In each window the local offset of the vendor image was measured by amplitude cross-correlation and compared with the displacement that the planar-wavefront model of Appendix E predicts at the window center. The vendor image was then resampled through the model with a cubic kernel and, after a local realignment and phase-ramp removal in each window, the coherence and the log-amplitude correlation of the two images were computed. As delivered, the vendor image differs from the reference in two respects: it is masked to the parallelogram over which the vendor’s processor has full spectral support, and bright points carry cross-shaped sidelobes that the reference’s Taylor weighting suppresses.

The vendor image (Table 11) is displaced from the reference by a field whose largest value over the Panama windows is 13.2 pixels in azimuth and 9.0 in range. The model predicts 13.2 and 9.1 from the collection geometry alone. The root-mean-square difference between measured and predicted displacement over the windows is 0.11 and 0.12 pixel on Panama and at most 0.34 pixel on the other two collections, 1 to 4% of the largest displacement. After the resampling the root-mean-square residual displacement is at most 0.25 pixel (individual windows up to 0.43). The coherence of the two images averages 0.46, 0.67 and 0.63 on Panama, Melbourne and Iowa (0.67, 0.76 and 0.80 as the median over windows), and their log amplitudes correlate at 0.49 to 0.57 within windows (Figure 6).

At the brightest isolated points of the lock and port regions (the ship is a moving target and the vegetated region has no isolated point) the impulse-response widths of the two images differ by 1 to 27%. The vendor’s response is wider in azimuth at both points and, in range, narrower at one and 1.3% wider at the other, which is consistent with a different aperture weighting in the vendor’s processor. The SICD metadata does not state whether autofocus was applied, and the vendor’s

motion compensation is independent of the one used here. Coherence at this level is attributed mainly to the differences between the two processors and is not used as a measure of precision. The vendor image is delivered without a correction for the displacement; the polar-format images of this study remove it in the final resampling (Appendix E).

The Capella 2021 stripmap and 2024 spotlight of Section 5.4 were compared with the vendor’s image on the vendor’s pixel grid by the same method. The correlation of linear amplitudes is 0.993 for the stripmap mosaic (165 patches, 512 by 512 pixel crop) and 0.78 for the spotlight formed by exact backprojection (128 by 128 pixels at the scene center); neither figure is comparable with the windowed log-amplitude correlation of Table 11. In both of these CPHD files the phase sign declared in the metadata is opposite to the sign with which the data focus, and FastSAR’s reader reverses it.

G Comparison with open-source implementations

RITSAR [17] (commit 0e36d2e, ported to Python 3) is a NumPy library. **bpBasic** is the backprojection of the AFRL toolbox of Gorham and Moore [2], distributed by the National Geospatial-Intelligence Agency (commit 63f0401); it was run in GNU Octave 8.4. ISCE3 [19] (0.25.17; 0.25.21 for the Capella sliding spotlight) has C++/OpenMP and CUDA versions, and torchbp [20] (commit b371884) is a PyTorch extension with CUDA kernels. GRDL [18] (commit de3338a) is a Python library with Numba kernels that reads CPHD files and implements polar format and fast factorized backprojection, run with its default parameters.

All implementations received the Taylor-windowed phase history and antenna positions of the reference (Appendix A) in their own input formats and were scored with the error measure of Section 4 on three of the regions of Figure 3, ISCE3 on 256 by 256 pixel patches of its own radar grid. The whole instance was available to every run; ISCE3 kept about 5.6 of the 16 vCPUs busy. The scripts, records and a longer account of each comparison are in the directory **results/comparison** of the study repository.

RITSAR’s error arises from two indexing defects. Its range axis is built with **linspace**, so the spacing is too large by one part in $N_{\text{fft}} - 1$, and its frequency axis is centered on the assumption of an even sample count, which the 14,399 samples of the collection do not satisfy. The error of **bpBasic** falls by 12 to 15 dB per doubling of its FFT length, as linear interpolation of the range profiles predicts. At 2^{17} it stays 15 to 17 dB above FastSAR’s exact backprojection with linear interpolation of profiles oversampled 8 times (region errors in the study records). This gap is attributed to **bpBasic** referencing the phase of its profiles to the lowest frequency of the band.

ISCE3’s amplitude correlates with the reference at 0.999 or more; most of its complex difference is a linear phase ramp common to all patches, of the kind that a difference in spectral centering would produce. Once the ramp is removed, the center patch agrees to -54.9 dB and the off-center patches retain an unexplained residual of -31.5 to -31.9 dB.

The torchbp kernels evaluate the distance in float32 (Table 3, footnote); its exact backprojection forms no image, whereas a float64 evaluation of the same formula does. Its factorized backprojection required more than 23 GiB, which exceeds the 22.0 GiB that PyTorch reports as the L4’s capacity. GRDL’s factorized backprojection, which its documentation describes as a stripmap algorithm, ran on the CPU in 77.5 s. Its image of the spotlight collection is smeared in cross-range, and no orientation of its grid registers to the reference (amplitude correlation 0.22 or less).

The AFRL toolbox also contains a polar format, **pfa_mem**, run unmodified in Octave on the same instance (with a compatibility version of **cross** for mixed row and column inputs), and GRDL’s polar format ran on the same instance. Neither image is corrected for the planar-wavefront displacement. Each region was therefore registered and resampled onto the reference pixels before

Table 12: Polar-format implementations on the Panama collection, on the CPU instance: correlation of the log amplitude with the reference and mean coherence in 5 by 5 pixel windows over the three regions, and the time per full image (s).

Implementation	Log-amplitude correlation	Coherence	Time per full image
AFRL toolbox <code>pfa_mem</code> (Octave)	0.80 to 0.89	0.76 to 0.88	44.5 s
GRDL <code>PolarFormatAlgorithm</code>	0.78 to 0.93	0.83 to 0.94	24.5 s
FastSAR polar format, float32	0.996 to 0.999	0.999	11.4 s

scoring by amplitude statistics and coherence, as for the vendor image of Appendix F (Table 12); part of their coherence loss comes from that resampling.

H Measurement protocol in detail

Software and builds. The software comprised FastSAR release 0.1.0 or the release candidates below, JAX 0.11.2 [34], CUDA 12 with CuPy 14.2.0 [40], and Numba 0.68.0 [30] for the reference. The Umbra records were made at the release-candidate tags `rel-rc3-20261009` (CPU and TPUs) and `rel-rc5-20261009` (L4), the Capella records at `rel-rc2-20261009` (CPU and v6e) and `rel-rc5-20261009` (L4). At `rel-rc2` and `rel-rc3` the library code differs from release 0.1.0 only in the CUDA path and the GPU memory settings, which the CPU and TPU records do not exercise, and at `rel-rc5` only in one docstring.

The exact backprojection records (Tables 3, 4 and 14, Figure 1 and Appendix C) were remade at `rel-rc6-20261009` (L4, TPUs) and `rel-rc7-20261009` (CPU). Their library code differs from release 0.1.0 in the default profile oversampling of `ExactFormer`, 8 instead of 4 times. At `rel-rc7` it also includes the fixes to reading long collections listed in the changelog of release 0.1.1, which the Umbra records do not exercise; release 0.1.1 contains all of these changes. The Umbra records name their commits, and the Capella records’ logs do. Every configuration ran in its own process through the public interface and is the fastest of the implementation variants whose errors agreed within 0.1 dB in development.

Time and cost. For the factorized algorithm the timed interval of Section 4 includes the per-collection host work (the float64 geometry of all three levels for the CUDA and C++ kernels and of the first two for the TPU kernels). Timing followed the two-warm-call rule of Section 4; the first warm call was up to 32% slower where one-time work fell on it (90th percentile of the difference 5%), and on the CPU and the L4 the reported time agrees with the loop’s time per image within 6%.

The loop ran until at least four images had been formed and at least 60 s had elapsed, forming images in sequence on the CPU and the L4 and, on the TPUs, staging the next phase history on a worker thread (`ImageFormer.stage`, as in the mosaics) while the current image forms; on the CPU, exact backprojection and the JAX program were timed without a loop. The factorized kernels were tuned against the bounds of Table 9; polar format was not tuned, and its costs are upper bounds. The L4 was hosted by a g2-standard-4 instance for FastSAR on the Umbra collections, a g2-standard-8 for the open-source implementations and a g2-standard-32 for the Capella collections. The process’s CPU time was recorded during every run.

Power. The L4’s draw was sampled from `nvidia-smi` twice a second during the timed loops. The TPU virtual machines and the CPU instance expose no power counters; the TPU ranges, derived

Table 13: Energy per Panama image for the processor alone, over the single-image time (the kWh column of Table 4 uses the per-image time of the timed loop, which is shorter than the single-image time on the TPUs only). The L4 draw is the mean of `nvidia-smi` samples during the timed loop; the TPU figures are ranges based on design-power figures quoted by third parties (120 to 200 W for the v5e, 200 to 350 W for the v6e [43, 44]), since the announcement of the v6e [38] gives no power figure, assuming the chip ran at that power throughout; the CPU figure is the pro-rata share of the 8 cores (25 W of the 400 W package) times their occupancy, the busy fraction of the 16 hardware threads doubled and capped at one. Hosts, memory and the other components of each instance are excluded.

Device	Configuration	Basis	Power (W)	Time (s)	Energy (kJ)
L4	Exact BP, cubic interpolation, $8\times$ oversampled	measured	71	24.1	1.73
L4	Factorized BP, float16 throughout, CUDA kernels	measured	60	3.5	0.21
L4	Factorized BP, float32, CUDA kernels	measured	64	3.8	0.24
TPU v5e	Factorized BP, single-pass products, fused kernels	third-party design power	120 to 200	3.6	0.4 to 0.7
TPU v5e	Factorized BP, three-pass products, fused kernels	third-party design power	120 to 200	5.2	0.6 to 1.0
TPU v6e	Factorized BP, single-pass products, fused kernels	third-party design power	200 to 350	2.6	0.5 to 0.9
TPU v6e	Factorized BP, three-pass products, fused kernels	third-party design power	200 to 350	3.1	0.6 to 1.1
CPU	Factorized BP, float32, C++ kernels	pro-rata estimate	25 at 100%	11.3	0.28

from third-party design-power figures, and the pro-rata CPU estimate are in Table 13 and are marked as estimates.

Precision. The gain fitted per algorithm family is a complex gain for the backprojections and a real one for polar format, whose phase differs from the reference by a smooth screen before the final resampling. Six statistics are computed for each image. The error is the energy of the difference relative to the energy of the reference (dB); this is the figure quoted when a single number is given. The second is the largest single-pixel difference relative to the brightest pixel (dB). The third is the complex coherence between the two images over a 5 by 5 window, reported as the mean, the 0.1 percentile and the minimum. The fourth and fifth are the amplitude ratio and the phase difference over the brighter half of the pixels, reported as the 99th percentile and the maximum. The sixth is the error in four rings by distance from the scene center.

Table 15 gives the main statistics of the Panama configurations, and the supplementary tables give all six for every configuration on every collection.

CPU details. On the CPU the factorized float32 row comes from the C++ kernels of Appendix D and exact backprojection from its C++ kernel; polar format runs in pure JAX. The reference was formed on a separate n2-highmem-48 instance (48 vCPUs, \$3.14 per hour on demand) in 1892 s per Panama image (about \$1,652 per thousand images) and is not part of the cost comparison.

Reference convergence and exact backprojection. The reference’s convergence was measured on a 1024 by 1024 crop by forming the same crop at 8, 16 and 32 times. The 8 times image differs from the 32 times image by -55.5 dB and the 16 times image by -69.2 dB. The 16 times reference was adopted because the 8 times convergence error, -55.5 dB, lies within the range of the errors under study. On the lock, port and ship regions the same float64 code with profiles oversampled 64 times (the 64-times computation of Section 4) differs from the reference by -67.3 to -68.8 dB, the reference’s own error.

FastSAR’s exact backprojection with linear interpolation of 16 times oversampled profiles, the

interpolation of the reference, measures -80.5 dB against it; the shared interpolation error cancels, and the figure indicates only agreement between the two implementations. With linear interpolation the error is set by the profile oversampling, -56.9 dB at 8 times. Against the 64-times computation, cubic interpolation at 8 times oversampling, the default of release 0.1.1, gives -77.5 to -81.7 dB in the three regions, and cubic at 4 times, the default of release 0.1.0, -59.8 to -69.6 dB. Linear interpolation of oversampled profiles is the scheme of the open-source implementations (Table 3 gives each one’s oversampling).

On 512 by 512 pixel grids at the same regions, cubic interpolation at 16 times and linear at 64 times are within 0.3 dB of cubic at 8 times, so more oversampling does not lower the error; the float32 arithmetic and the third-order range expansion of the kernel are the likely source of this floor. With linear interpolation at 16 times oversampling the L4 takes 26.9 s per image, about \$5.3 per thousand images.

Limitations of the cost comparison. The improvements still identified in the factorized kernels are about 12% on the v6e and 10% on the L4 in the profiled builds, and none was quantified on the CPU, whose stages run at 1.05 to 1.4 times their bounds (Appendix D). The factorized plan (32-pixel final tiles, three levels, a passband fraction of 0.4 and 70 dB filter attenuation) kept its defaults on every device; a different factorization would change the bounds themselves, by an amount not measured here. TPU7x and H100 instances were outside the project’s cloud quota. The cost of the asterisked row of Table 4 is computed from the single-image time, not from a loop. The TPU hosts (24 and 44 vCPUs) are larger than the work needs, while the L4’s (4 vCPUs) is the smallest offered. Only on-demand prices were used; spot and committed-use prices were not considered.

I Ground truth of the Panama regions

The identity of the Panama regions used in Figure 3 was checked against Copernicus Sentinel-2 imagery [28]. Each pixel of the images formed here was located on the ground from the vendor’s SICD header. The pixel’s position in the slant plane follows from the scene center point and the row and column unit vectors. The ground point is the intersection of the line through it along the slant plane’s normal with the WGS84 ellipsoid at the scene height. The image corners so computed agree with the corner coordinates in the SICD header to within 15 m.

The nearest usable Sentinel-2 acquisition is from 2023-07-23, five days later, at 10 m resolution; a dry-season acquisition of 2024-02-23 is shown as well because clouds cover part of the earlier one. The lock chambers, the quays and container yard of the terminal, and the road beside the vegetation region coincide in the two sensors.

The bright target of the ship region, about 180 m long and close to the eastern bank, does not appear in the vendor’s geocoded products of six other passes of this repeat-collection site. Four of those passes are three, four, five and seven days after this collection and two are in February 2024. The pass of 2023-07-25 02:30 UTC repeats the geometry of this collection to within 0.3° (azimuth 122.4° versus 122.1° , grazing angle 42.6° in both, left-looking). This makes an aspect-dependent return from a fixed object unlikely. The other passes span azimuths from -72° to 223° and grazing angles from 40° to 63° .

The pass of 2023-07-23 14:47 UTC precedes the Sentinel-2 image of that day by 64 min, and neither shows anything at the position. It is taken to be a ship in the navigation channel; the Sentinel-2 image of 2024-02-23 shows a different ship nearby. The vendor’s geocoded product of this collection places the target inside the outlined crop as well.

The ship appears on the eastern bank, inland of the channel, as a consequence of its motion

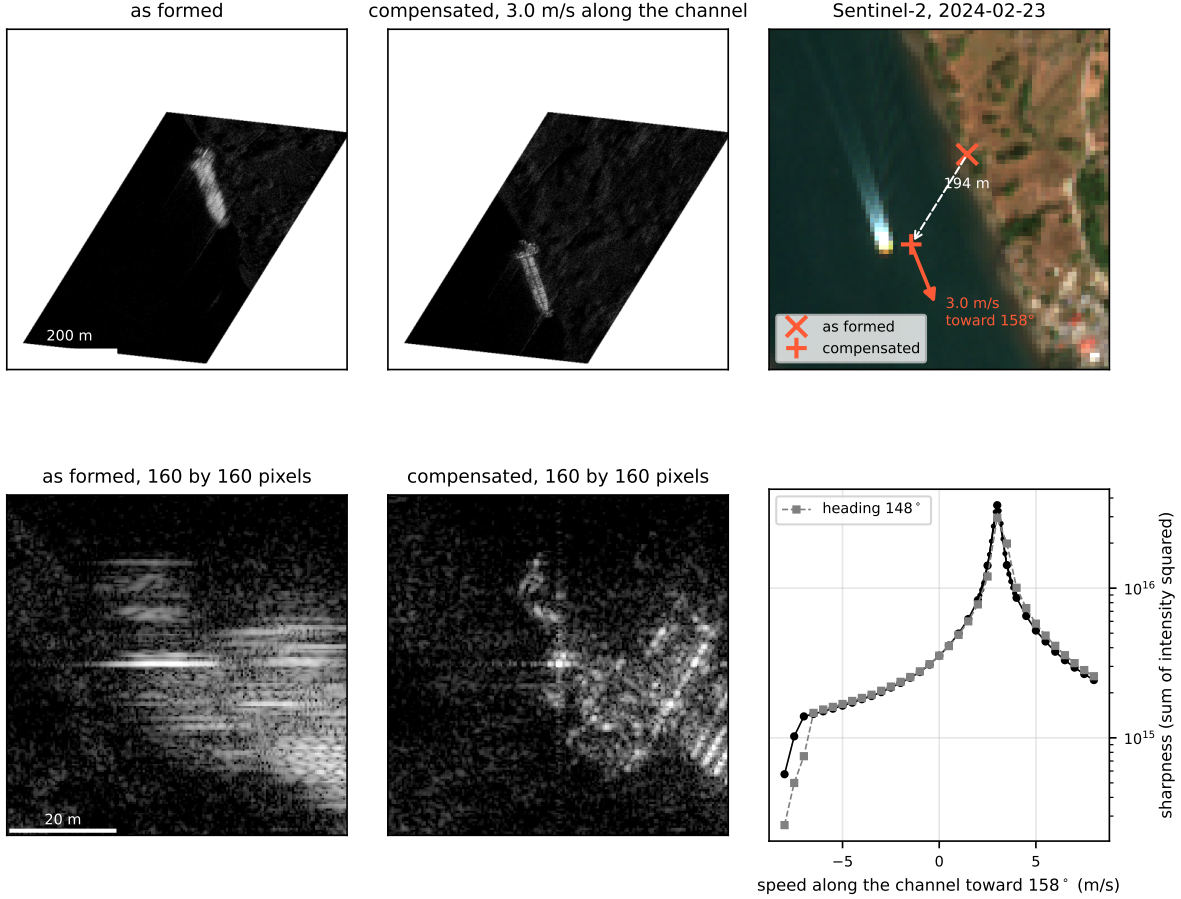


Figure 7: The ship compensated for its motion. Top: the tile around the ship as formed and after backprojection with the antenna track shifted for a ship moving at 3.0 m/s along the channel, both resampled north up over a 760 m window (200 m bar), and the two positions on the Sentinel-2 image of 2024-02-23, with the 194 m displacement (dashed) and the estimated velocity, 3.0 m/s toward 158°, drawn at the compensated position. Bottom: 160 by 160 pixel windows around the ship in image coordinates (20 m bar), and the sharpness of the tile against the assumed speed for the channel heading of 158° (coarse and fine sweeps) and for 148°.

during the aperture. A stationary-scene processor places a target with line-of-sight velocity v_r at the position whose Doppler history matches, displaced along azimuth by about Rv_r/V (here about 100 m per m/s), and the along-track velocity defocuses it [45]. Exact backprojection takes the antenna position of every pulse as data, so a target moving at constant velocity $\mathbf{v}$ is formed exactly by backprojecting with the track shifted by $-\mathbf{v}t$. This both focuses the target and places it at its position at the aperture center.

The ship was assumed to follow the channel axis (bearing 158°, from the Sentinel-2 water mask) and its speed was swept from -8 to 8 m/s in a tile around it (Figure 7). The sharpness of the tile, the sum of the squared intensities, is single-peaked at 3.0 m/s (5.8 knots toward the Pacific). At that speed the peak intensity is 10.6 dB above the stationary-scene image and the ship has moved 469 pixels (194 m) along azimuth into the navigation channel; the position of the water was not used in the estimate. A heading 10° farther east (148°) gives the same speed with an 18% lower

Table 14: Melbourne and Iowa: time, throughput and cost for the configurations of Table 4 except exact backprojection on the CPU; bold marks the fastest configuration on each collection (in Table 4 bold marks the Pareto front).

Device	Algorithm	Arithmetic	Time (s)	Images/h	\$ per 1000	kWh per 1000
<i>Melbourne</i>						
L4	Factorized BP	float16 throughout, CUDA kernels	3.6	1,004	0.70	0.06
L4	Factorized BP	float32, CUDA kernels	3.9	901	0.78	0.07
L4	Polar format	float32, gather	3.1	1,156	0.61	0.04
L4	Exact BP	cubic interpolation, 8× oversampled	26.6	135	5.24	0.53
L4	Exact BP	linear interpolation, 8× oversampled	20.6	178	3.98	0.40
TPU v5e	Factorized BP	single-pass products, fused kernels	3.7	1,462	0.82	0.08 to 0.14
TPU v5e	Factorized BP	three-pass products, fused kernels	5.5	863	1.39	0.14 to 0.23
TPU v5e	Polar format	float32, gather	68.2	53	22.75	2.28 to 3.79
TPU v6e	Factorized BP	single-pass products, fused kernels	2.6	2,377	1.14	0.08 to 0.15
TPU v6e	Factorized BP	three-pass products, fused kernels	3.1	1,791	1.51	0.11 to 0.20
TPU v6e	Polar format	float32, gather	106.5	34	79.87	5.92 to 10.35
CPU	Factorized BP	float32, C++ kernels	12.8	285	3.38	0.09
CPU	Polar format	float32, gather	11.9	299	3.23	0.08
<i>Iowa farmland</i>						
L4	Factorized BP	float16 throughout, CUDA kernels	2.3	1,578	0.45	0.04
L4	Factorized BP	float32, CUDA kernels	2.5	1,418	0.50	0.05
L4	Polar format	float32, gather	2.3	1,595	0.44	0.03
L4	Exact BP	cubic interpolation, 8× oversampled	18.1	200	3.53	0.36
L4	Exact BP	linear interpolation, 8× oversampled	12.6	281	2.52	0.25
TPU v5e	Factorized BP	single-pass products, fused kernels	2.5	2,173	0.55	0.06 to 0.09
TPU v5e	Factorized BP	three-pass products, fused kernels	3.5	1,342	0.89	0.09 to 0.15
TPU v5e	Polar format	float32, gather	48.3	75	16.10	1.61 to 2.68
TPU v6e	Factorized BP	single-pass products, fused kernels	1.8	3,329	0.81	0.06 to 0.11
TPU v6e	Factorized BP	three-pass products, fused kernels	2.2	2,538	1.06	0.08 to 0.14
TPU v6e	Polar format	float32, gather	61.1	59	45.85	3.40 to 5.94
CPU	Factorized BP	float32, C++ kernels	7.3	477	2.02	0.05
CPU	Polar format	float32, gather	8.4	424	2.27	0.06

sharpness peak and a compensated position 22 m away. The images compared in Figures 3 and 4 are the stationary-scene images, in which the ship appears at the same displaced position under every algorithm; the motion analysis was used only to identify the target.

J Precision and cost tables

The time and statistics of every configuration on each scene are in the supplementary tables of the study repository ([paper/supplement_tables.pdf](#)); the configuration labels are those of Section 3.

Table 15: Precision of the Panama Canal images against the float64 exact backprojection: energy of the difference (dB), largest pixel difference relative to the brightest pixel (dB), 0.1 percentile and minimum of the 5 by 5 coherence (the mean is in the supplementary tables), amplitude ratio (99th percentile and maximum, dB) and phase difference (99th percentile and maximum, degrees) over the brighter half of the pixels. The 16 times oversampled exact row uses the interpolation of the reference (Appendix H).

Device	Configuration	Error	Max	Coh. p0.1	Coh. min	Amp. p99	max	Phase p99	max
L4	Exact BP, cubic interpolation, 8× oversampled	−67.4	−72.1	1.0000	0.998	0.01	0.6	0.1	4
CPU	Exact BP, cubic interpolation, 8× oversampled	−67.4	−72.1	1.0000	0.998	0.01	0.6	0.1	4
L4	Exact BP, linear interpolation, 8× oversampled	−56.9	−62.2	0.9998	0.923	0.03	2.5	0.2	15
CPU	Exact BP, linear interpolation, 8× oversampled	−56.9	−62.2	0.9998	0.923	0.03	2.5	0.2	15
L4	Exact BP, linear interpolation, 16× oversampled	−80.5	−88.8	1.0000	1.000	0.00	0.1	0.0	0
L4	Factorized BP, float32, CUDA kernels	−59.9	−65.0	0.9999	0.931	0.02	4.2	0.1	20
L4	Factorized BP, float16 throughout, CUDA kernels	−58.9	−64.7	0.9999	0.932	0.03	4.2	0.2	20
CPU	Factorized BP, float32, C++ kernels	−60.0	−65.0	0.9999	0.931	0.02	4.2	0.1	20
CPU	Factorized BP, float32, JAX program	−58.3	−64.8	0.9999	0.931	0.03	4.2	0.2	20
TPU v6e	Factorized BP, three-pass products, fused kernels	−57.8	−64.5	0.9999	0.931	0.03	4.2	0.2	20
TPU v6e	Factorized BP, single-pass products, fused kernels	−46.2	−64.8	0.9981	0.674	0.16	10.4	1.1	106
TPU v5e	Factorized BP, three-pass products, fused kernels	−57.9	−64.8	0.9999	0.931	0.03	4.2	0.2	20
TPU v5e	Factorized BP, single-pass products, fused kernels	−46.2	−65.0	0.9981	0.674	0.16	10.9	1.1	106
L4	Polar format, float32, gather	−32.1	−40.3	0.9857	0.680	0.69	20.7	6.1	70
TPU v6e	Polar format, float32, gather	−32.1	−40.3	0.9857	0.680	0.69	20.7	6.1	70
TPU v5e	Polar format, float32, gather	−32.1	−40.3	0.9857	0.680	0.69	20.7	6.1	70
CPU	Polar format, float32, gather	−32.1	−40.2	0.9857	0.679	0.69	20.1	6.1	70